



Visual Recognition using Deep Learning

Convolutional Neural Networks & Object Recognition

林彥宇 教授

Yen-Yu Lin, Professor

國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Andrej Karpathy, Fei-Fei Li, Justin Johnson, Harsh Agrawal, and Alex Krizhevsky

Outline

- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



Outline

- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



Outline

- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



LeNet

- First convolutional neural networks (?)
- Proposed by Yann LeCun for digit recognition
- Network architecture of LeNet-5

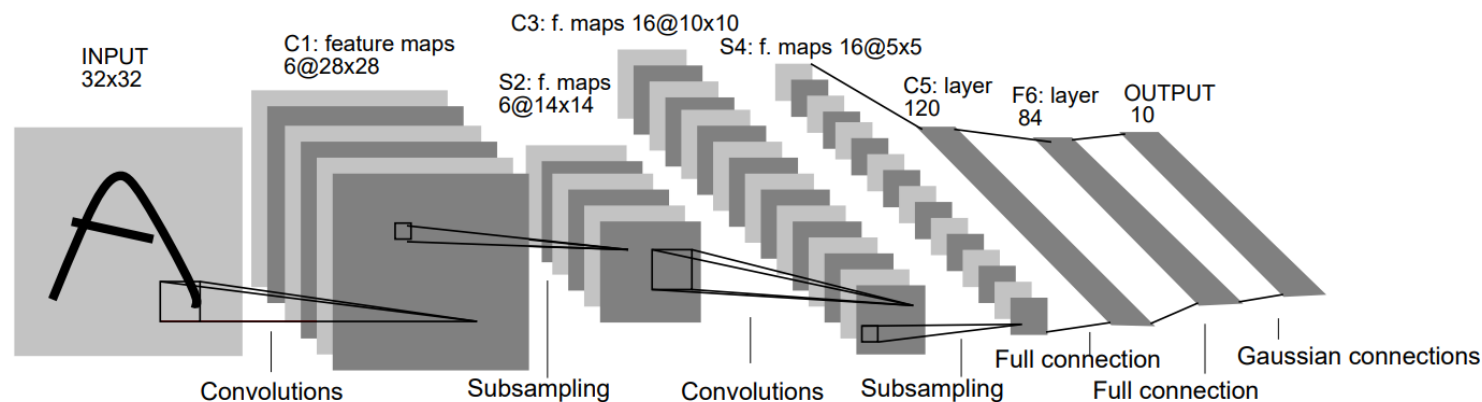


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- Three key ideas: Local receptive fields, weight sharing, sub-sampling (pooling)

LeNet-5: Overview

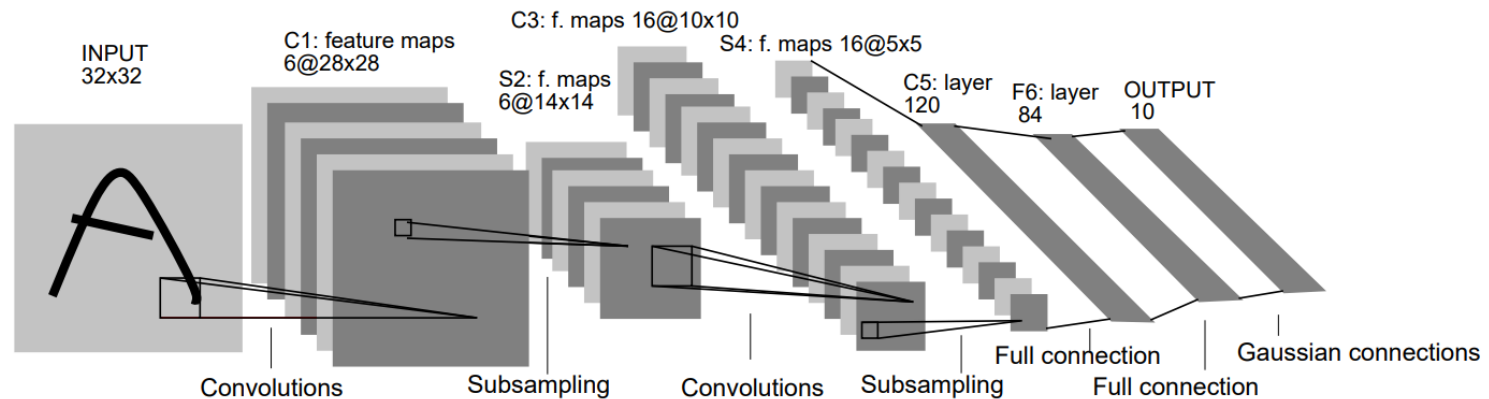


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- Input: A 32x32 image
- Black and white pixel values are normalized, e.g., white = -0.1, black = 1.175 (mean of pixels = 0, Std of pixels = 1)
- Network layers: conv->pooling->conv->pooling->FC->FC

LeNet-5: Layer C1

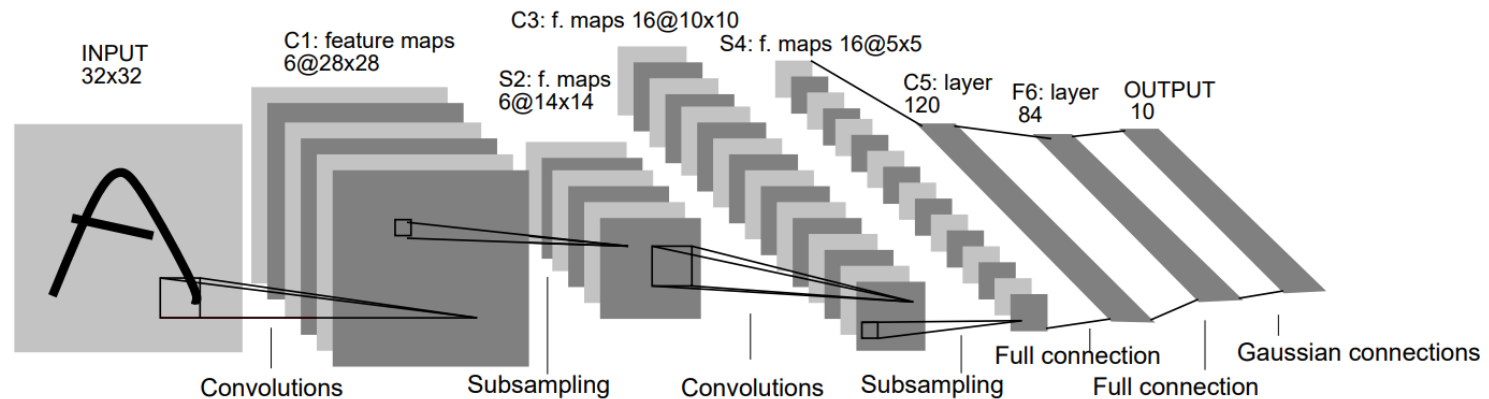


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- C1: Convolutional layer with 6 feature maps of size 28x28x6
- Each filter has a 5x5 receptive field in the input image
 - Local receptive field, sparse connections, and weight sharing
- $(5 \times 5 + 1) \times 6 = 156$ learnable parameters

LeNet-5: Layer S2

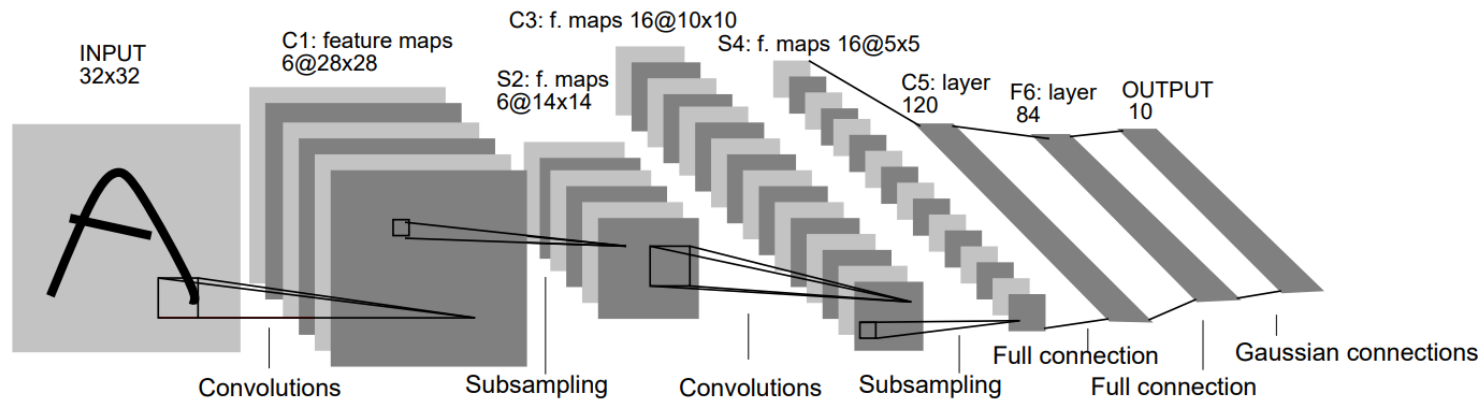


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- S2: Sub-sampling (pooling) layer
- 2x2 non-overlapping pooling windows
- 6 feature maps of size 14x14x6

LeNet-5: Layer C2

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

TABLE I

EACH COLUMN INDICATES WHICH FEATURE MAP IN S2 ARE COMBINED
BY THE UNITS IN A PARTICULAR FEATURE MAP OF C3.

- C2: Convolutional layer with 16 feature maps of size 10x10x16
- Each filter is of size 5x5 combining a subset of feature maps
- 1516 learnable parameters
 - Filters #0 ~ #5: $(5 \times 5 \times 3 + 1) \times 6 = 456$
 - Filters #6 ~ #14: $(5 \times 5 \times 4 + 1) \times 9 = 909$
 - Filter #15: $(5 \times 5 \times 6 + 1) \times 1 = 151$



LeNet-5: Layer S4

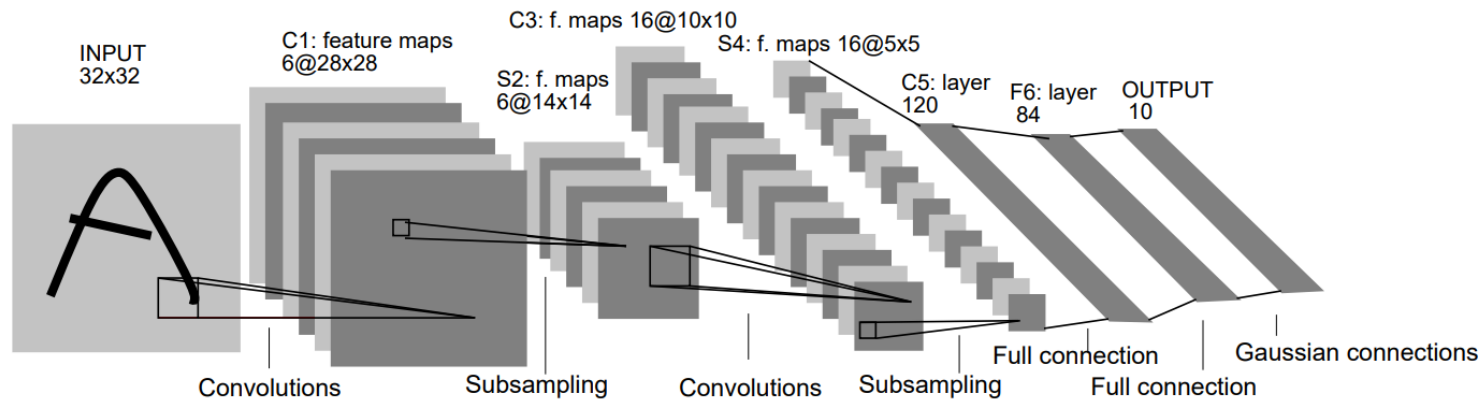


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- S4: Sub-sampling (pooling) layer
- 2x2 non-overlapping pooling windows
- 16 feature maps of size 5x5x16

LeNet-5: Layer C5

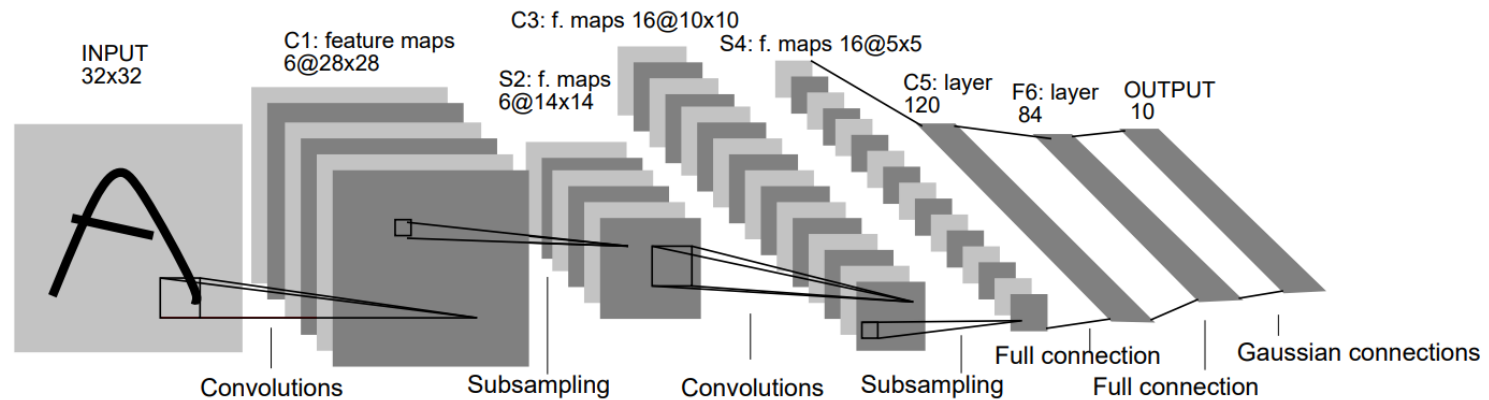


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- C5: Fully connected layer
- 120 neurons, each of which is connected to all 5x5x16 elements in the previous layer
- $(5 \times 5 \times 16 + 1) \times 120$ learnable parameters

LeNet-5: Layer F6

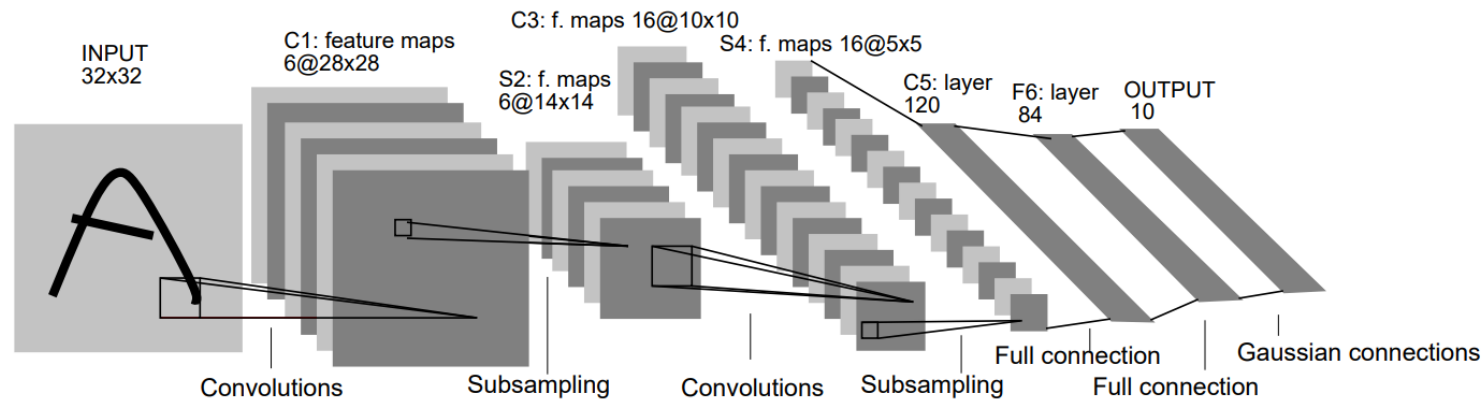


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- F6: Fully connected layer
- 84 neurons, each of which is connected to all 120 elements in the previous layer
- $(120+1) \times 84$ learnable parameters

LeNet-5: Output layer

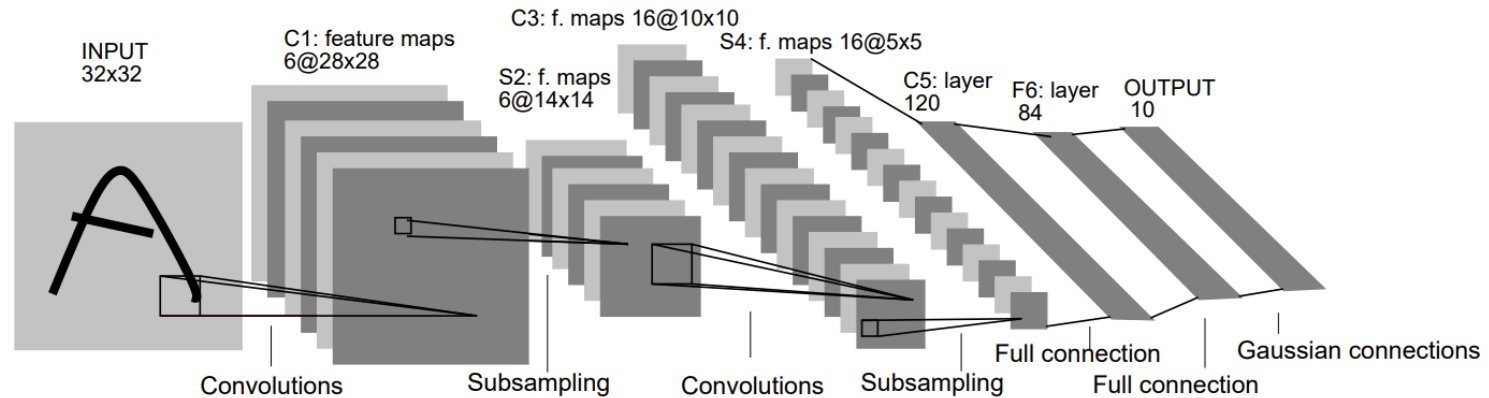
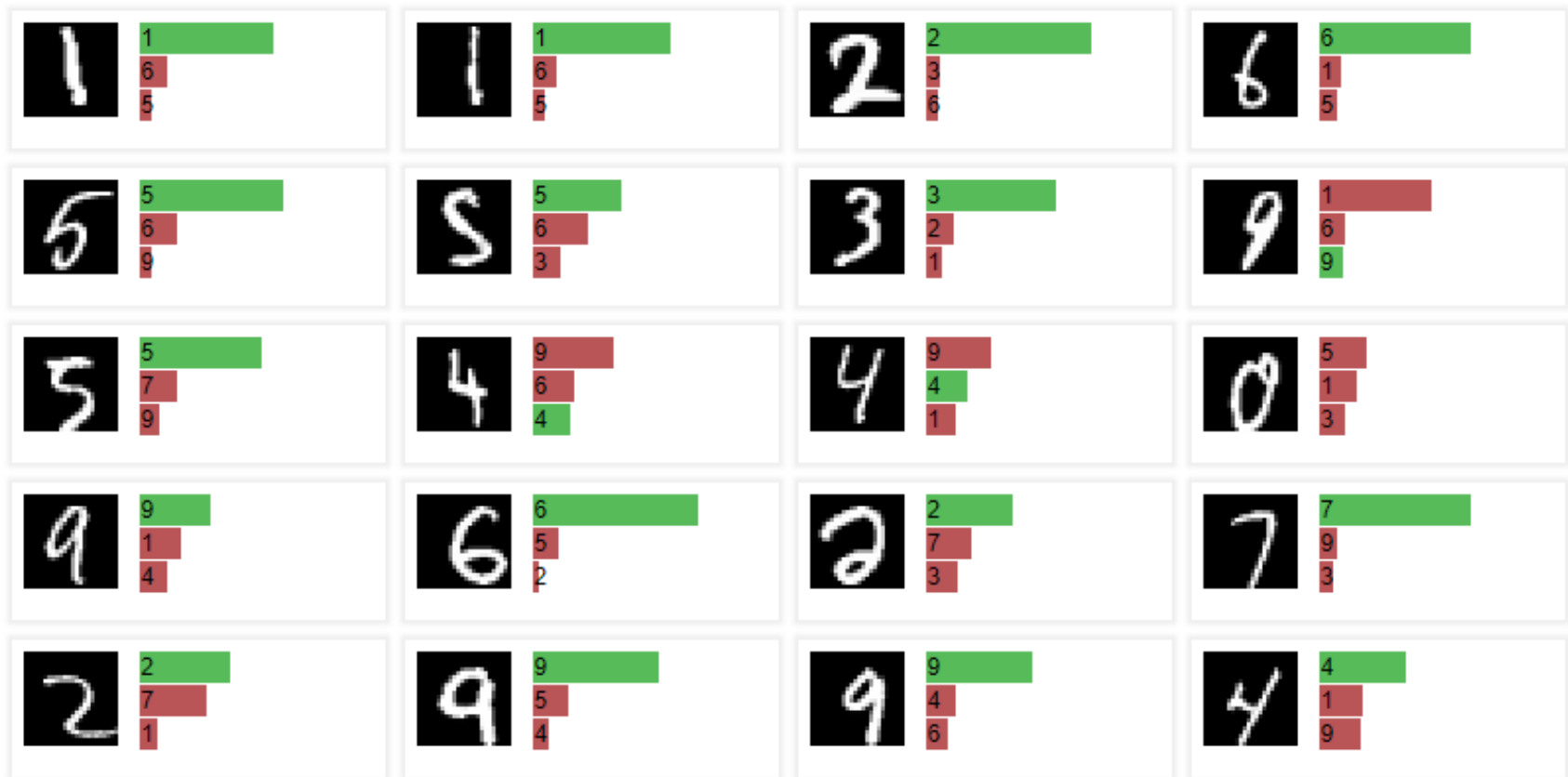


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

- F6: Fully connected layer
- 84 neurons, each of which is connected to all 120 elements in the previous layer
- $(120+1) \times 84$ learnable parameters
- 10 RBF units, one for each digit, are learned in the output layer

Some examples on the MNIST dataset



<https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html>



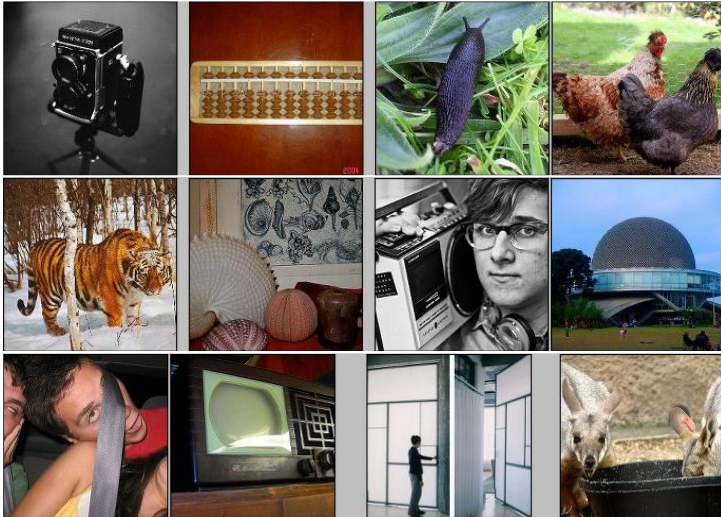
Outline

- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



ImageNet challenge

IMAGENET



- About 15M labeled high resolution images
- About 22K categories
- Collected from web and labeled by Amazon Mechanical Turk
- ImageNet Large-Scale Visual Recognition Challenge (ILSVRC):
 - 1.2 million training images of 1000 classes
 - 50K validation images
 - 150K testing images



ILSVRC winners

- The annual “Olympics” of computer vision.
- Teams from the world compete to determine who has the best computer vision models for classification, detection, segmentation, and more.
- The winner in 2012, AlexNet, achieves a top-5 error rate of 15.3%
- The next best team achieves an error of 26.2%



AlexNet

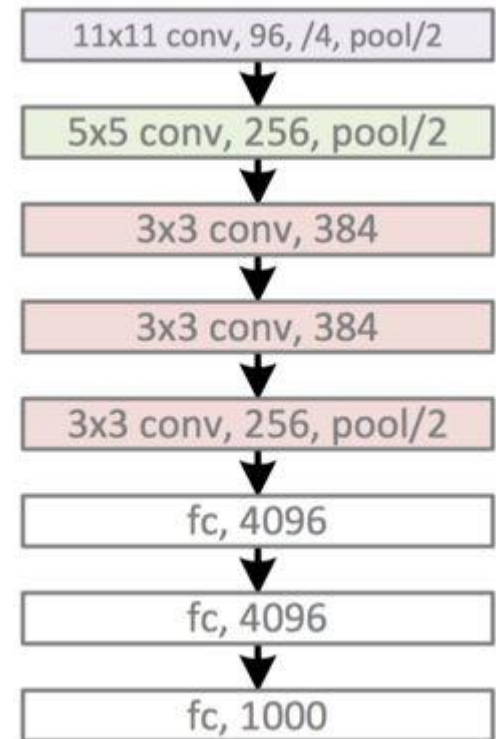
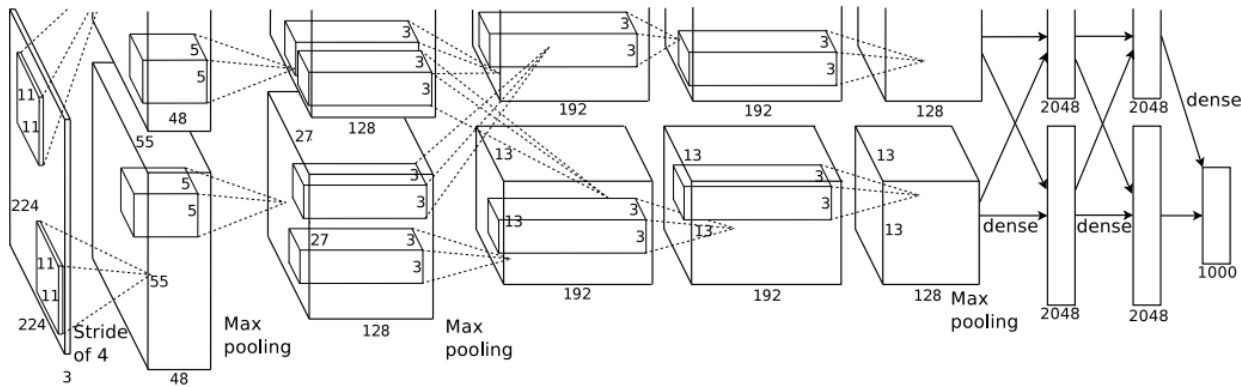
[Krizhevsky et al., NIPS'12]

- Similar framework to LeNet-5 but with the following differences:
 - Bigger model (5 convolutional and 3 fully connected layers)
 - 60M learnable parameters
 - More training data (10^6 vs. 10^3)
 - GPU implementation (50x speedup over CPU)
 - Trained on two GPUs for a week
 - Better regularization for training (DropOut)



AlexNet: Architecture

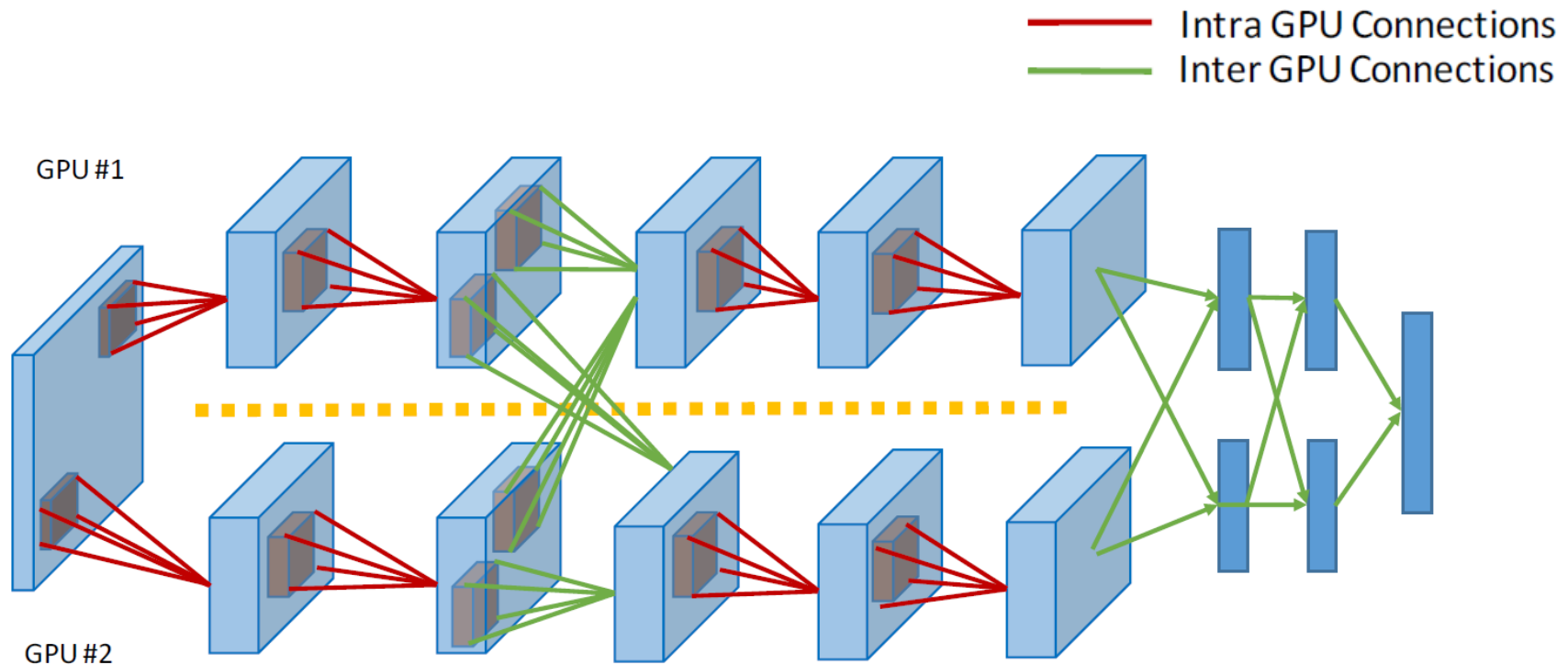
- Architecture overview
 - Five convolutional layers
 - Three fully connected layers
 - 1000-way softmax



AlexNet: Architecture

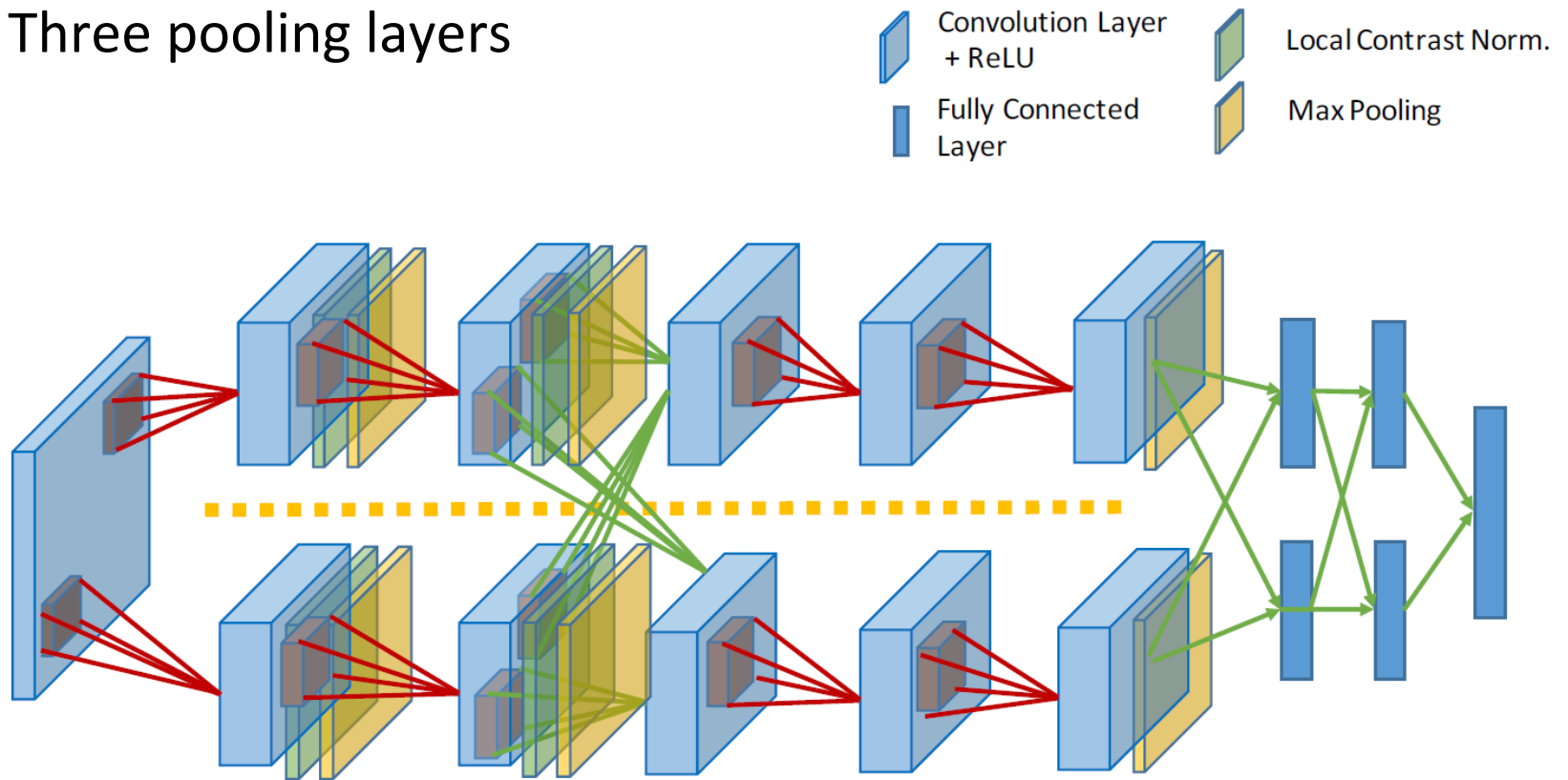
- Learning with two GPUs

- The 1st, 2nd, 4th, and 5th layers have intra-GPU connections
- The 3rd, 6th, 7th, and 8th layers have inter-GPU connections



AlexNet: Architecture

- Two normalization layers
- Three pooling layers



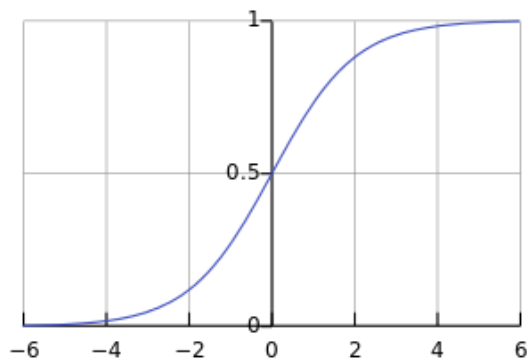
AlexNet: Contributions

- Five major contributions:
 - Nonlinearity: Rectified Linear Units (ReLU)
 - Multiple GPUs
 - Local response normalization (LRN)
 - Overlapping pooling
 - Reducing overfitting: data augmentation and dropout

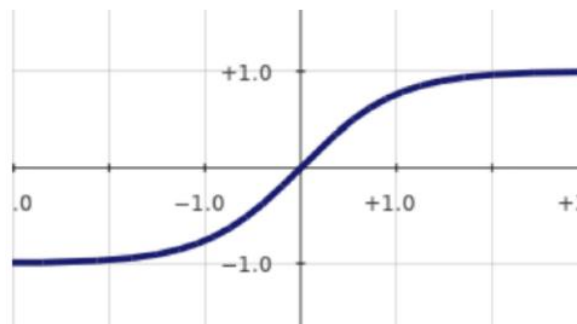


AlexNet: ReLU

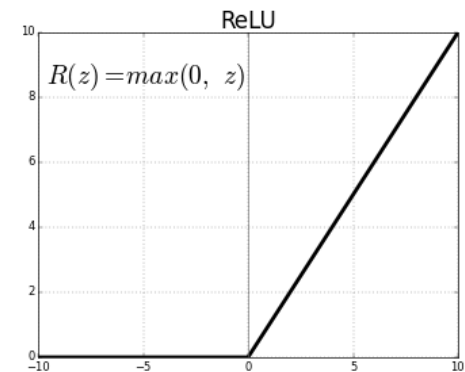
- Nonlinearity: Rectified Linear Units (ReLU)
 - To replace tanh or sigmoid
 - tanh: $f(x) = \tanh(x)$
 - sigmoid: $f(x) = (1 + e^{-x})^{-1}$
 - ReLU: $f(x) = \max(0, x)$
 - ReLU: Faster convergence and lower gradient vanishing



sigmoid function



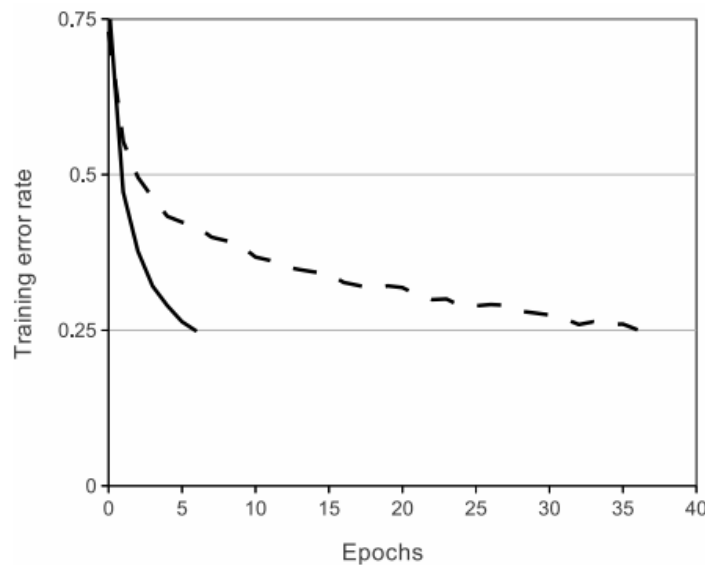
tanh



ReLU

AlexNet: ReLU and multiple GPUs

- With a four-layer CNN on the CIFAR-10 dataset, ReLU reaches the error rate of 25% about 7 times faster than tanh



ReLU (solid line)
tanh (dashed line)

AlexNet: Local response normalization

- Local response normalization (LRN)

$$b_{x,y}^i = a_{x,y}^i / \left(k + \alpha \sum_{j=\max(0, i-n/2)}^{\min(N-1, i+n/2)} (a_{x,y}^j)^2 \right)^\beta$$

normalized
response

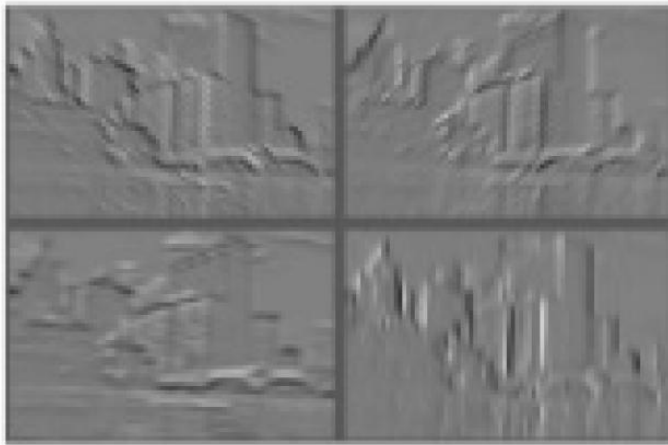
value of the i th feature
map at position (x,y)
before normalization

hyperparameter values:
 $k=2, \alpha=10^{-4}, \beta=-0.75$

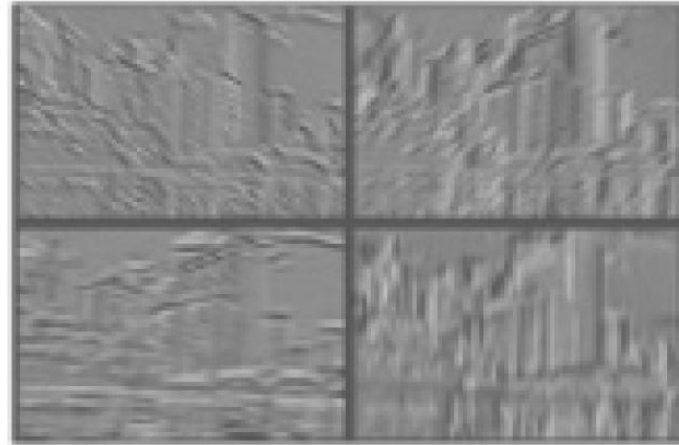


AlexNet: Local response normalization

- Reduce the top-1 and top-5 error rates by 1.4% and 1.2%



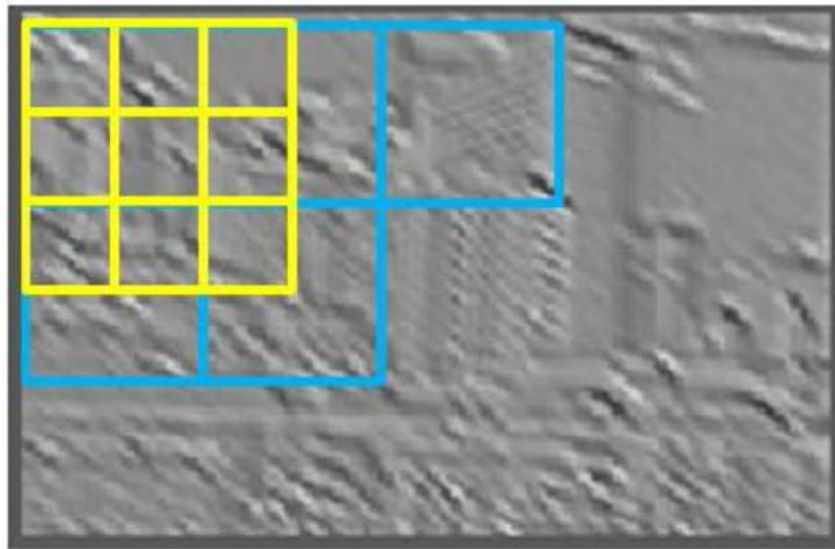
four feature maps before
normalization



normalized feature maps

AlexNet: Overlapping pooling

- Max pooling makes the resolution smaller and more manageable without losing too much information
- **Overlapping pooling**
 - Slightly alleviate the risk of overfitting
 - Reduce the top-1 and top-5 error rates by 0.4% and 0.3%



non-overlapping pooling
overlapping pooling

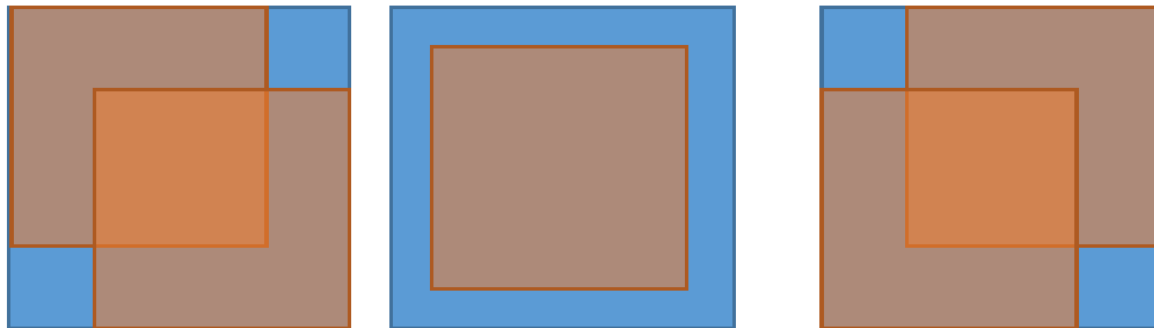
AlexNet: Data augmentation

- Overfitting: large performance gap between training and testing
 - A small number of training data
 - Complex training models
- AlexNet reduces overfitting: data augmentation and dropout
- Data augmentation
 - Easiest way to reduce overfitting on image data is to artificially enlarge the dataset using **label-preserving transformations**
 - Image translations and horizontal reflections
 - Altering the intensities of the RGB channels



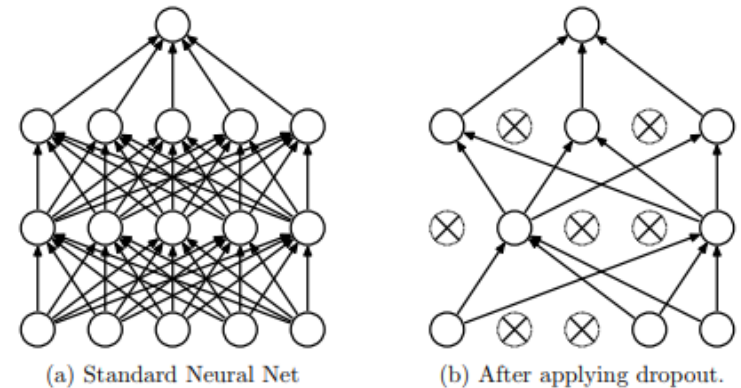
AlexNet: Data augmentation

- Data augmentation using translation and reflection
- **Image translation** by randomly extracting multiple 224×224 patches from a 256×256 image
- **Horizontal reflections** of these 224×224 patches
- At test time: 5 224×224 patches (four corners and one center patches) and their horizontal patches are used
- Averaging the predictions made by the softmax layer



AlexNet: Dropout

- Mini-batch optimization:
 - Randomly partition training data into batches
 - Forward propagation for response computation
 - Backward propagation for gradient computation
 - Update model
- For a batch of data, **dropout**
 - Apply to fully connected layers
 - Setting the output of each hidden neuron to zero with probability 0.5 during the training stage



[Srivastava et al., JMLR'15]

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

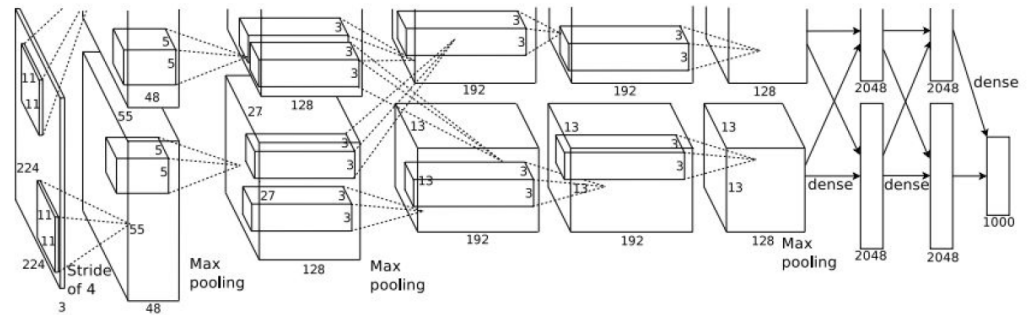
CONV5

Max POOL3

FC6

FC7

FC8



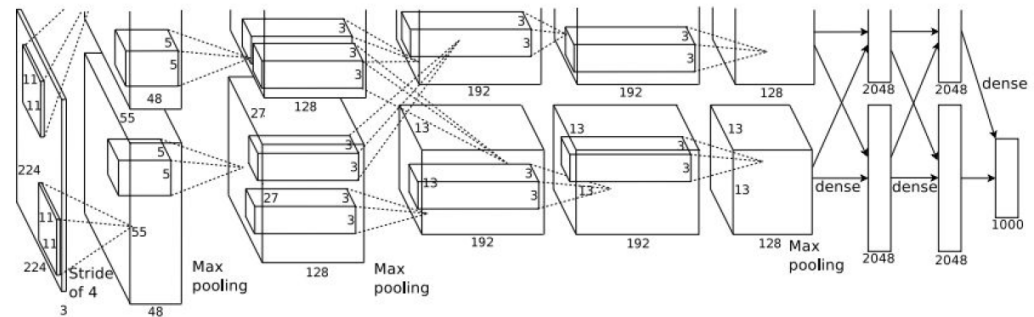
slide: Fei-Fei Li, Justin Johnson, Serena Yeung



AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

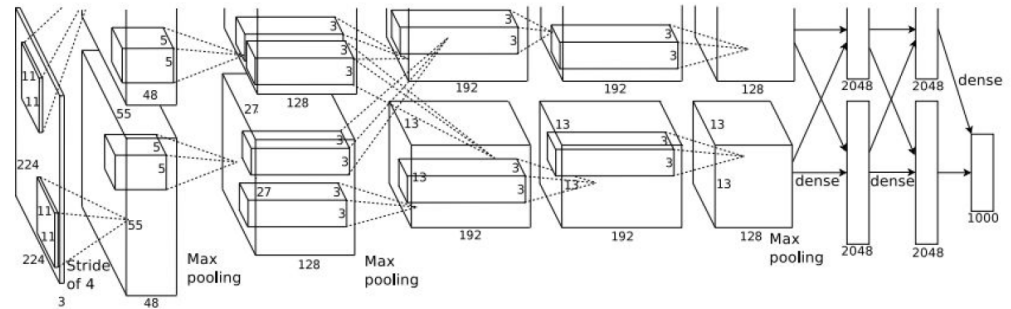
Q: what is the output volume size? Hint: $(227-11)/4+1 = 55$

slide: Fei-Fei Li, Justin Johnson, Serena Yeung

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

Output volume **[55x55x96]**

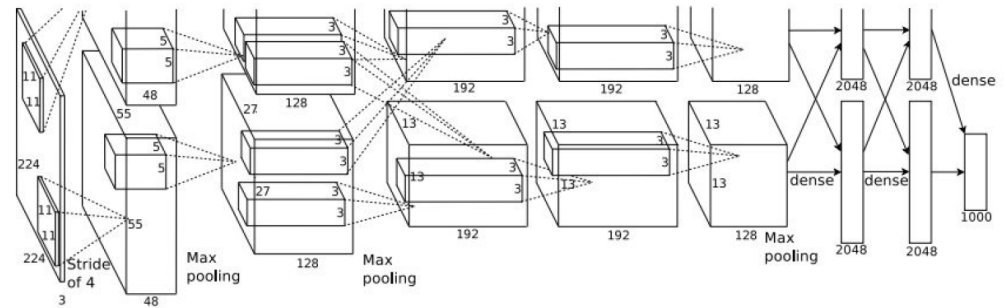
Q: What is the total number of parameters in this layer?

slide: Fei-Fei Li, Justin Johnson, Serena Yeung

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

Output volume **[55x55x96]**

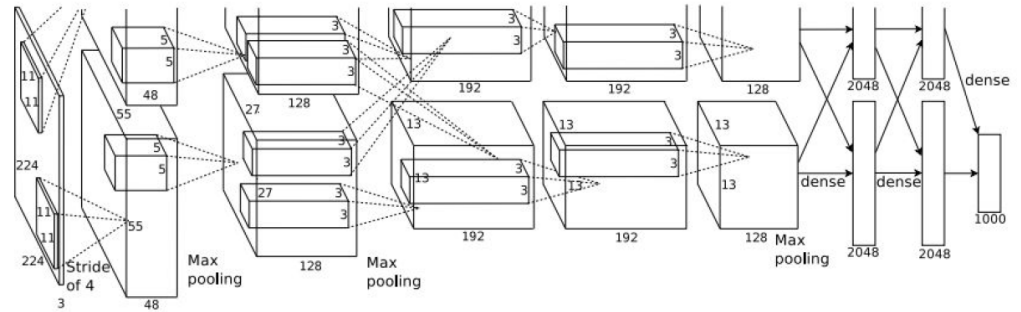
Parameters: $(11*11*3)*96 = 35K$

slide: Fei-Fei Li, Justin Johnson, Serena Yeung

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

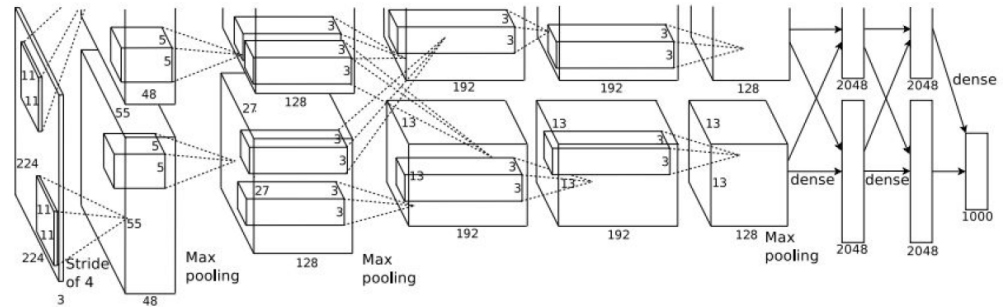
Second layer (POOL1): 3x3 filters applied at stride 2

Q: what is the output volume size? Hint: $(55-3)/2+1 = 27$

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

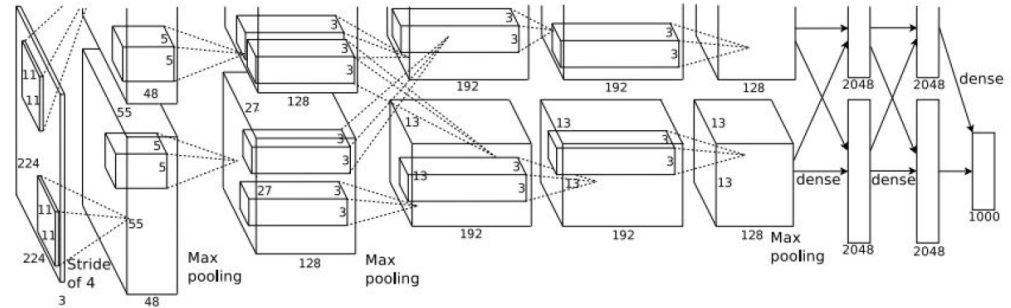
Q: what is the number of parameters in this layer?

slide: Fei-Fei Li, Justin Johnson, Serena Yeung

AlexNet

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

Parameters: 0!

slide: Fei-Fei Li, Justin Johnson, Serena Yeung

AlexNet: Experimental results

- ILSVRC-2012 competition
- Top-1 and top-5 errors

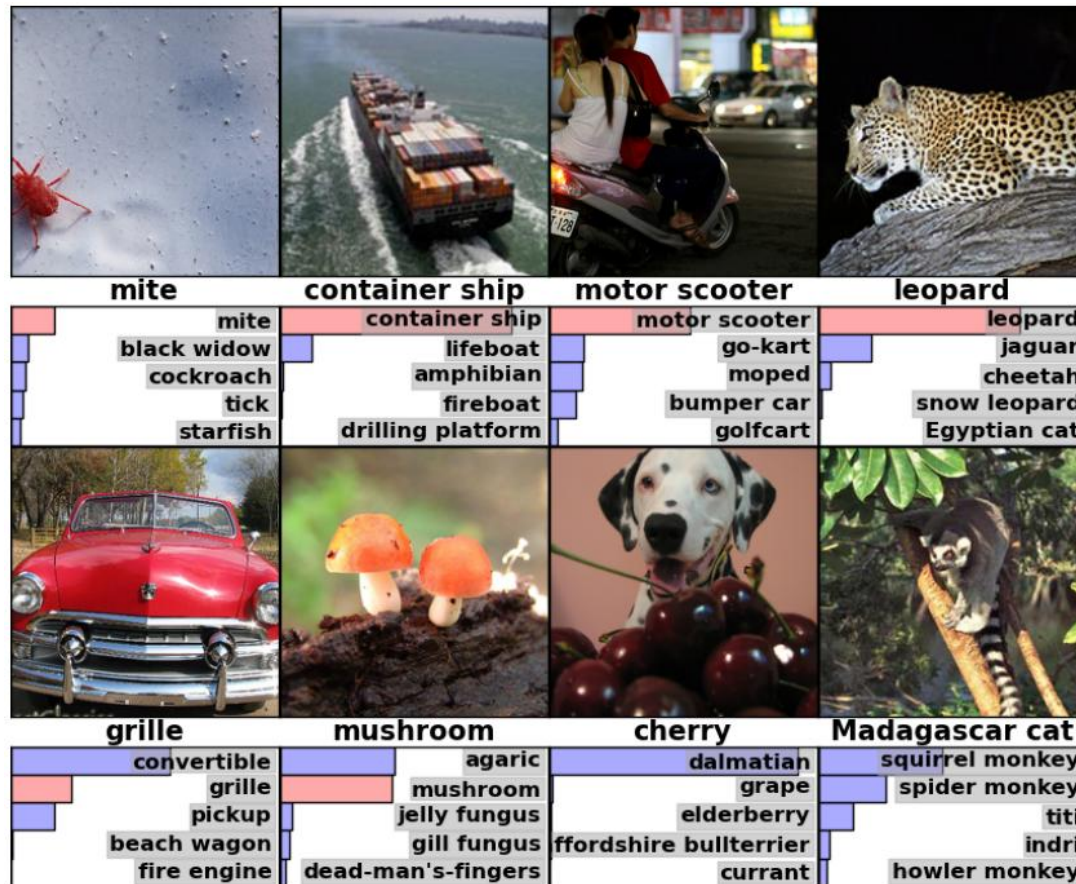
Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
<i>SIFT + FVs [7]</i>	—	—	26.2%
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	15.3%

- X CNNs: Average the predictions of X similar CNNs
- * denotes that models pre-trained to classify ImageNet 2011



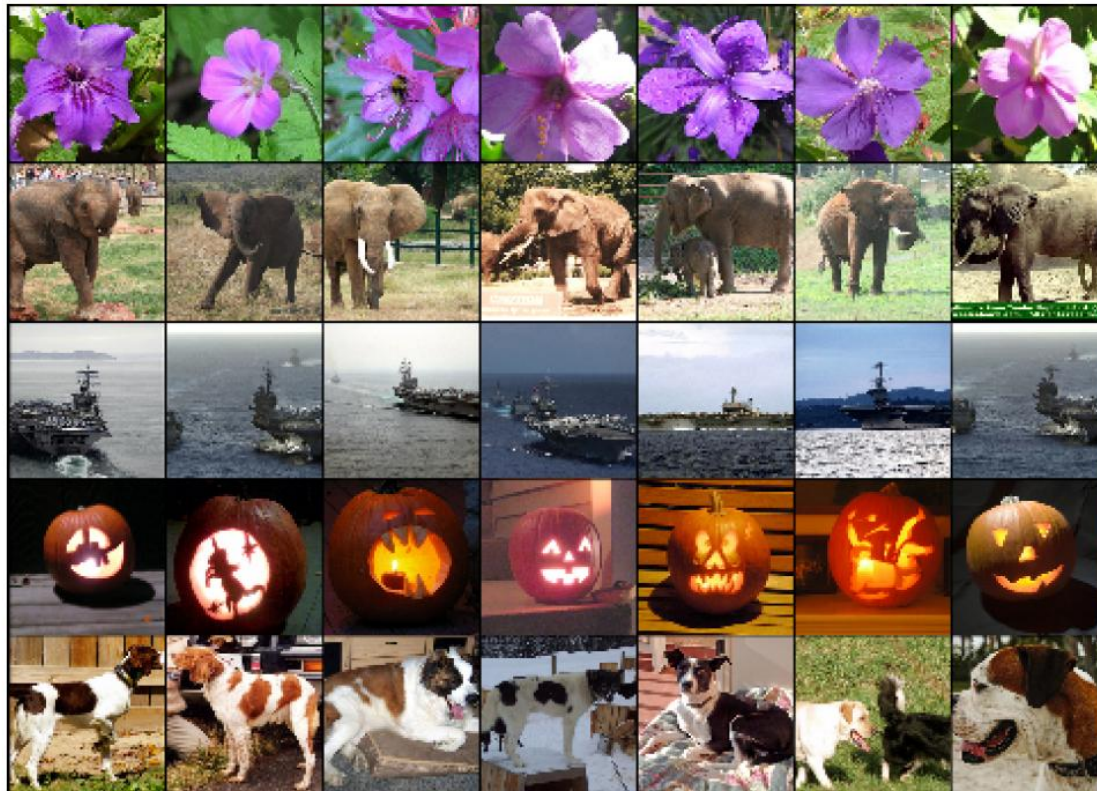
AlexNet: Experimental results

- Eight examples of top-5 prediction



AlexNet: Experimental results

- First column: Testing image
- The remaining columns: Its six nearest neighbors using deep features



Outline

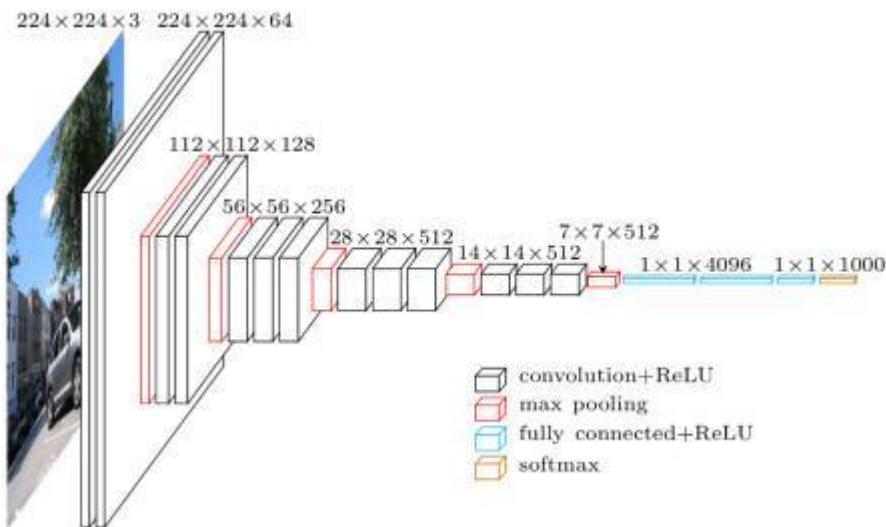
- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



VGG-Net

[Simonyan and Zisserman, ICLR'15]

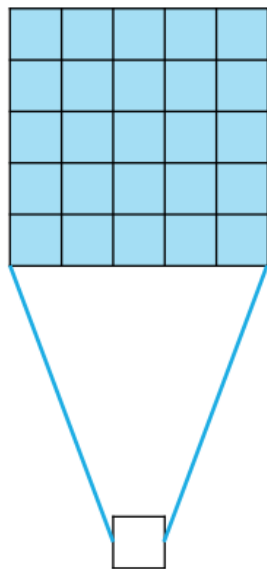
- Architecture overview
 - Effect of CNN depths on accuracy
 - 16 layers or 19 layers
 - Deeper than AlexNet
 - More accurate than AlexNet



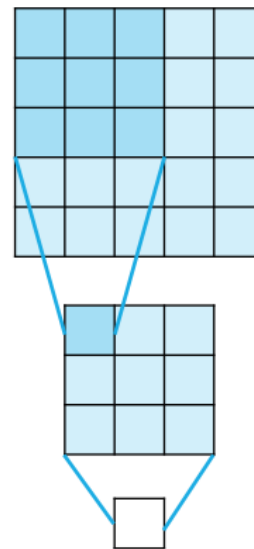
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG-Net

- 3x3 convolutional kernels – less parameters
 - Stacking convolutional layers have large **receptive fields**
 - More **non-linearity**
 - Less parameters to learn
 - More numbers of channels



- One 5x5 filter
- Parameters:
 $5 \times 5 = 25$
 - Non-linear:1



- Two 3x3 filters
- Parameters:
 $3 \times 3 \times 2 = 18$
 - Non-linear:2

Six network configurations

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					



Performance analysis

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	25.5	8.0

- A vs. A-LRN: LRN may not be effective
- A vs. B, C, D, E: The deeper, the better
- B vs. C: More 1 x 1 convolutional layers improve performance
- C vs. D: Spatial context is important
- Multiple scale training improves the performance



VGG-Net

- ILSVRC-2012 competition

Team	Year	Place	Error (top-5)	Uses external data	
SuperVision	2012	1st	16.4%	no	AlexNet
SuperVision	2012	1st	15.3%	Imagenet 22k	
Clarifai	2013	1st	11.7%	no	
Clarifai	2013	1st	11.2%	Imagenet 22k	
MSRA	2014	3rd	7.35%	no	
VGG	2014	2nd	7.32%	no	VGG-Net
GoogLeNet	2014	1st	6.67%	no	



Outline

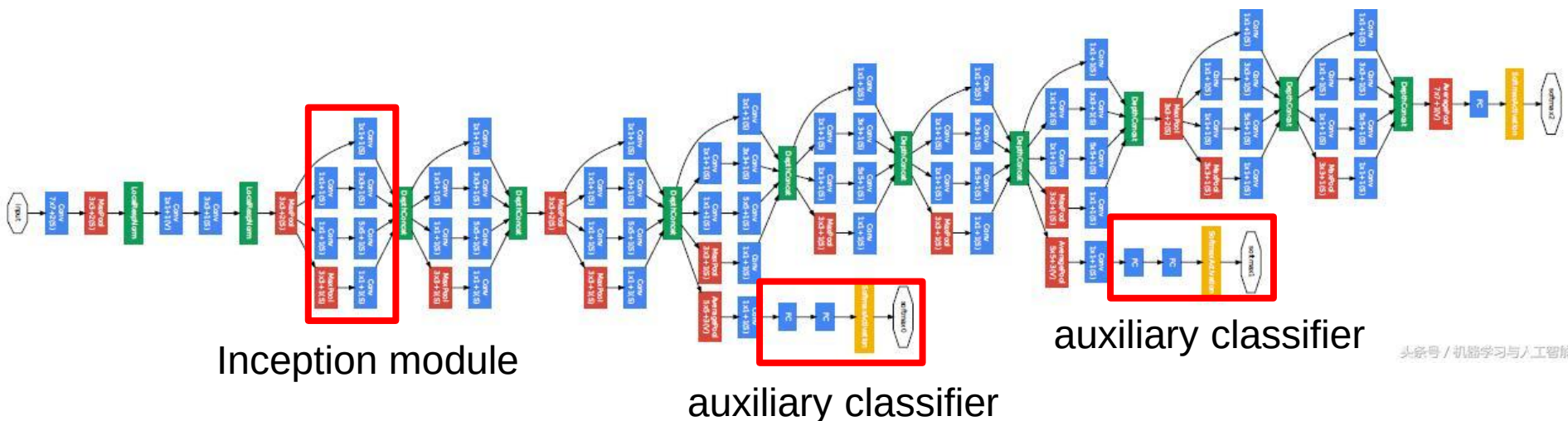
- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



GoogleNet (Inception V1)

[Szegedy et al., CVPR'15]

- Architecture overview
 - 22 layers
 - Significantly more accurate than AlexNet
 - 12 times lesser parameters than AlexNet
 - Inception module
 - Auxiliary classifiers

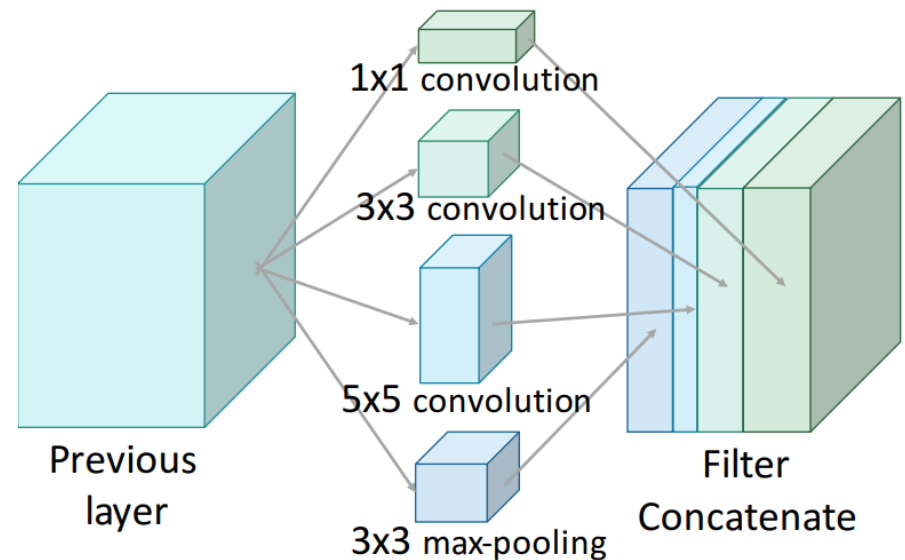
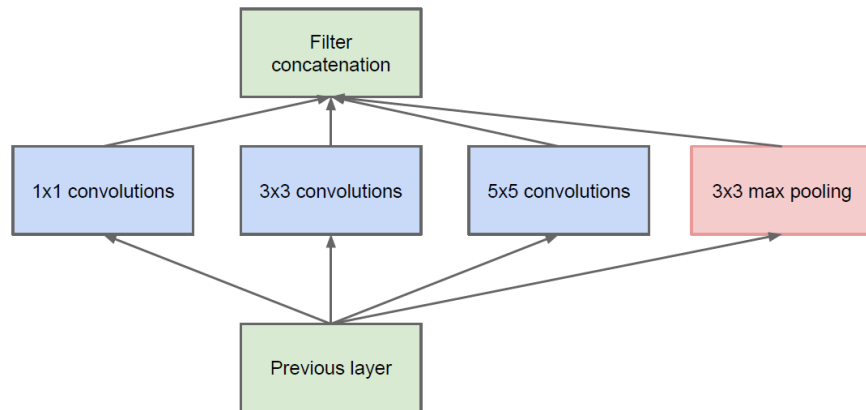


头条号 / 机器学习与人工智能



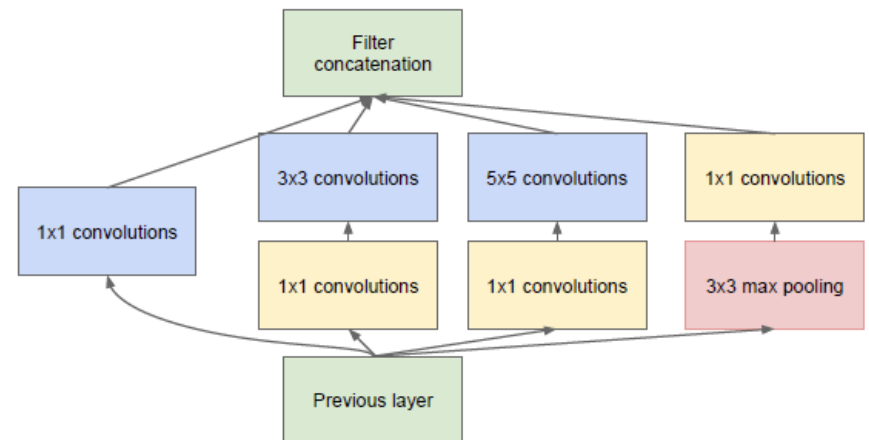
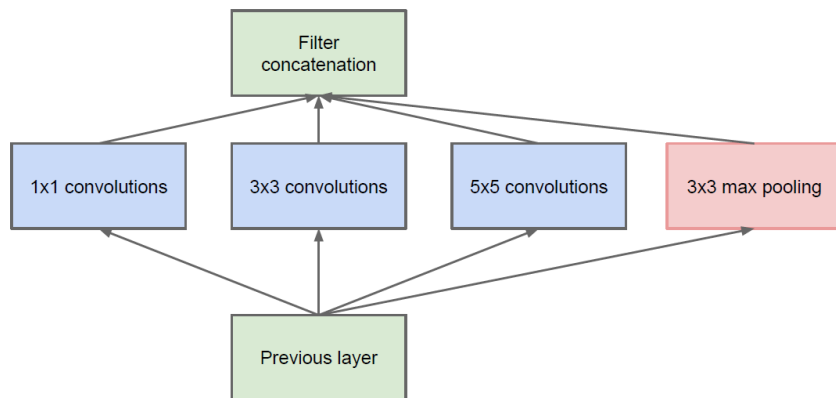
GoogleNet: Inception module

- Choose filter sizes of 1x1, 3x3, and 5x5
- Concatenate all feature maps
- Concatenate one additional pooling path, which is essential to the success of CNNs



GoogleNet: Inception module

- **Linear** decrease in feature maps results in **quadratic** decrease in computation
- Use inexpensive 1x1 convolutional layers to reduce the number feature maps



GoogleNet: Inception module stacking

- Stacking inception modules
- Conventional convolutional layers in lower layers
- Max pooling for size reduction

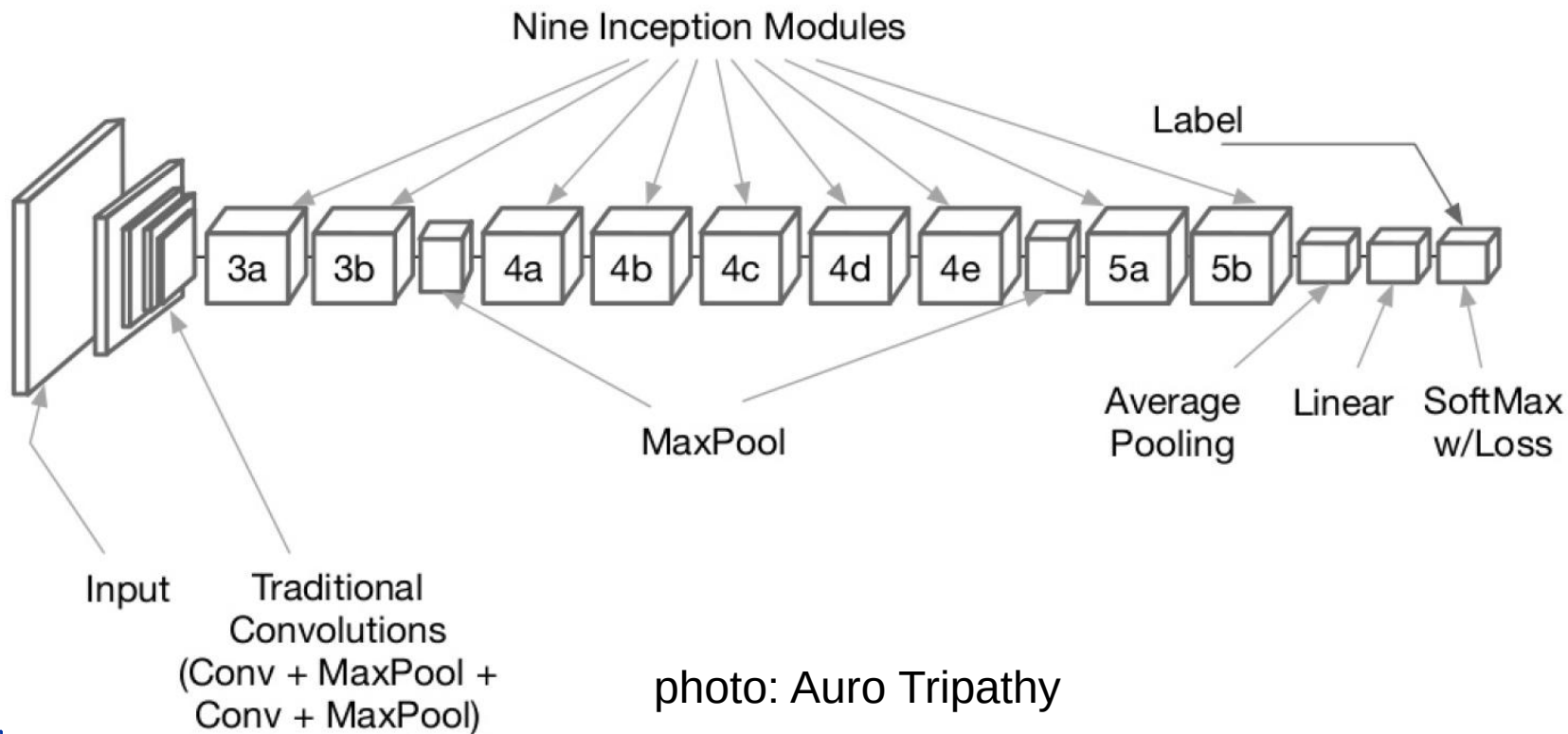
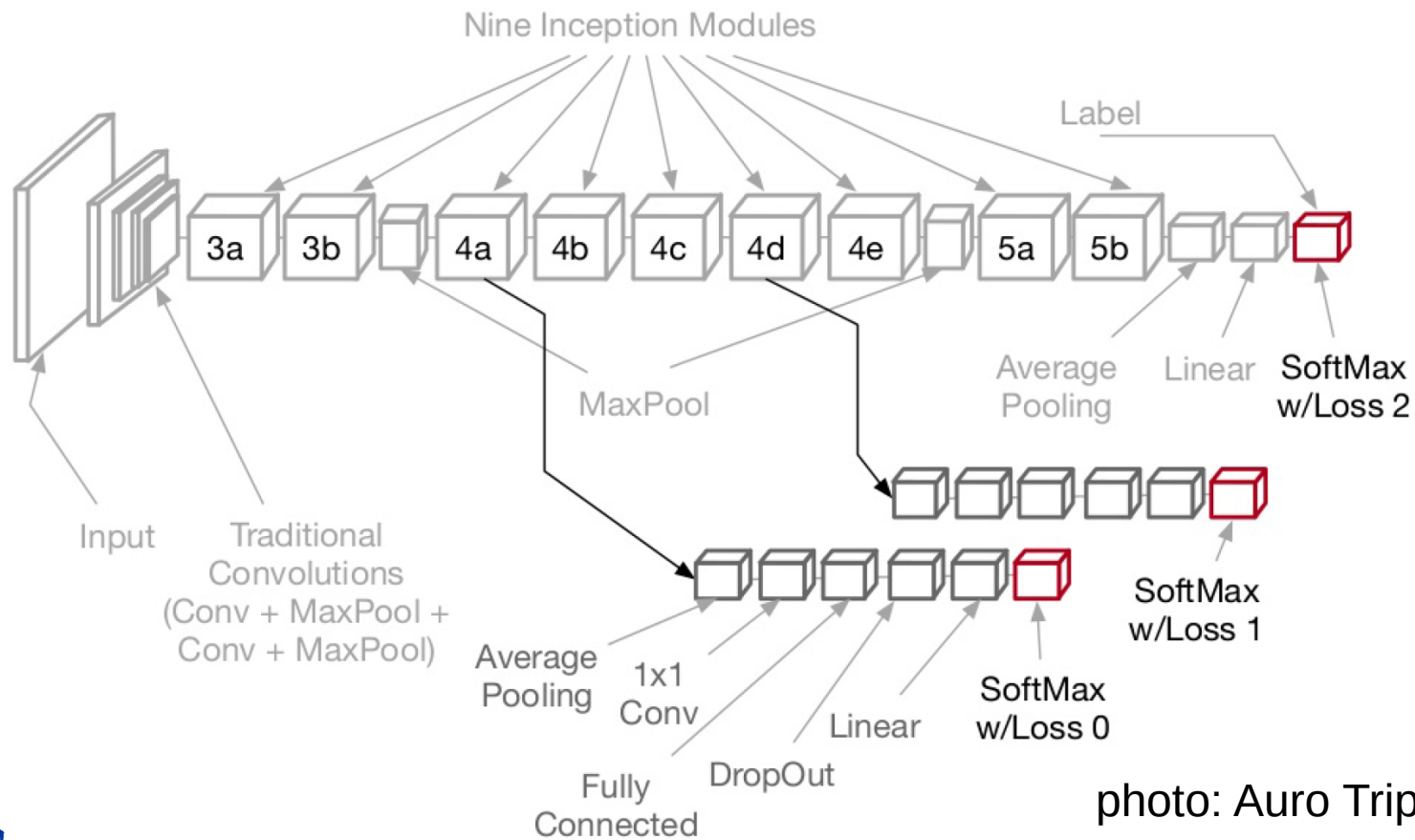


photo: Auro Tripathy

GoogleNet: Auxiliary classifier

- Deep networks result in **vanishing gradients**
- **Auxiliary classifiers** are appended



GoogleNet: Auxiliary classifier

- Intermediate layers become discriminative
- Intermediate loss alleviates the problem of vanishing gradient

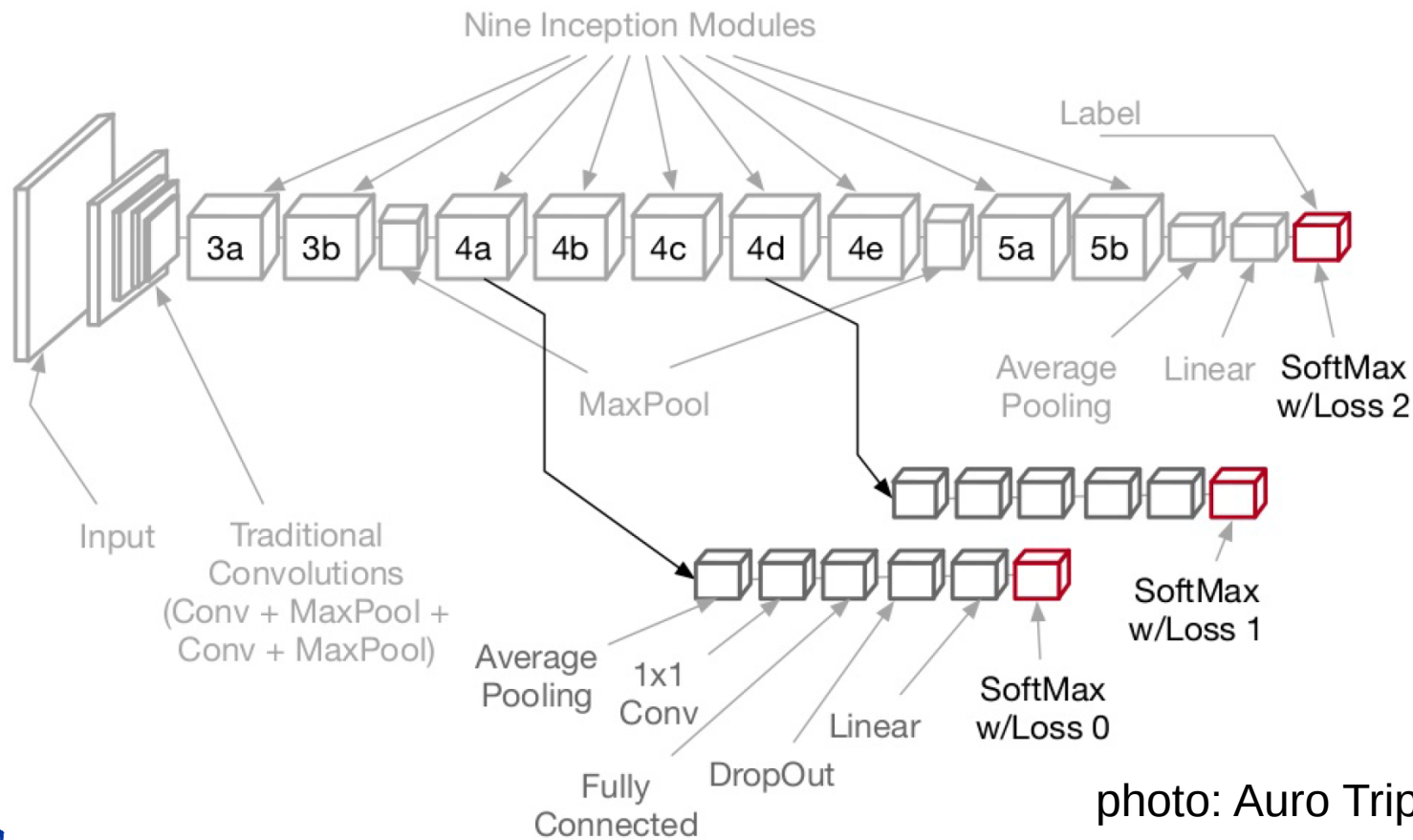


photo: Auro Tripathy

GoogleNet (Inception V1)

- ILSVRC-2012 competition

Team	Year	Place	Error (top-5)	Uses external data	
SuperVision	2012	1st	16.4%	no	AlexNet
SuperVision	2012	1st	15.3%	Imagenet 22k	
Clarifai	2013	1st	11.7%	no	
Clarifai	2013	1st	11.2%	Imagenet 22k	
MSRA	2014	3rd	7.35%	no	
VGG	2014	2nd	7.32%	no	VGG-Net
GoogLeNet	2014	1st	6.67%	no	GoogleNet



Outline

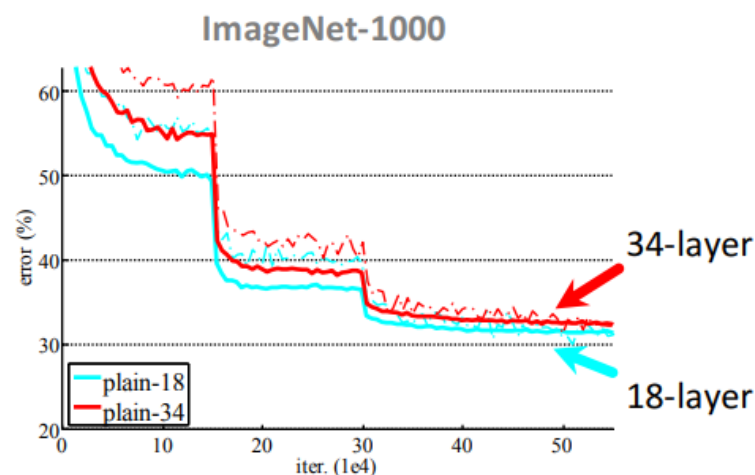
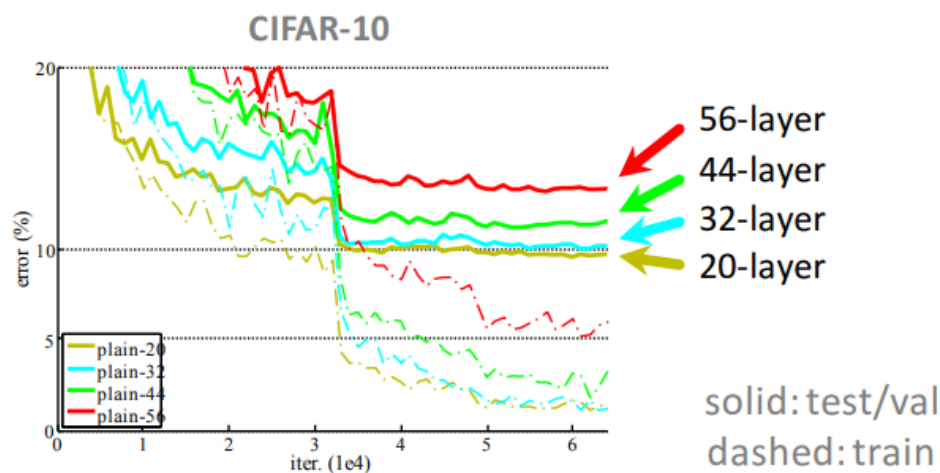
- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



ResNet

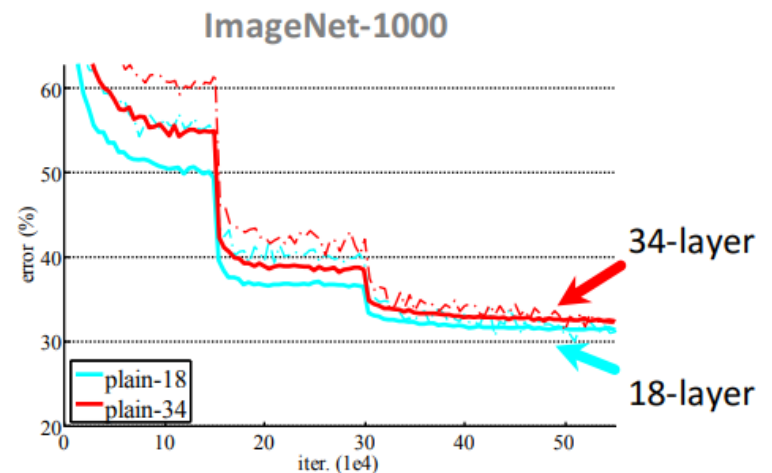
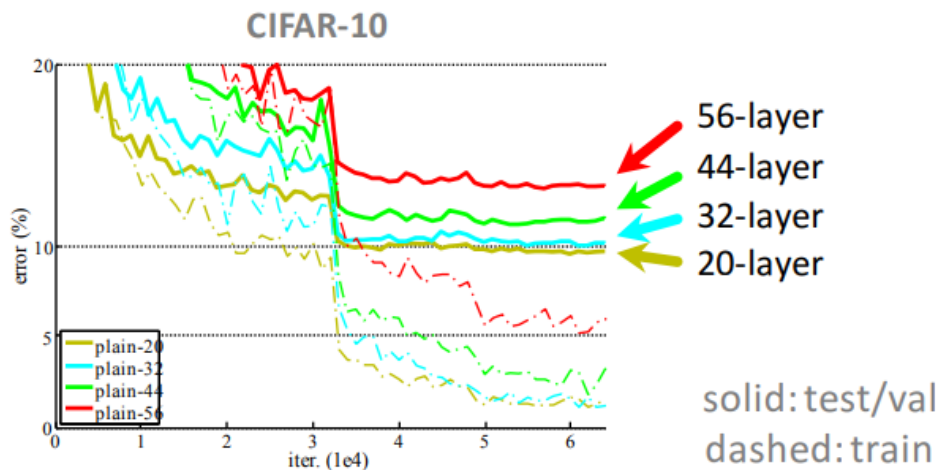
[He et al., CVPR'16]
Best Paper Award!

- The deeper, the better
 - Larger receptive field size
 - Higher non-linearity
 - Better fitting power
- Really?
 - Performance drops when using an overly deep CNN model



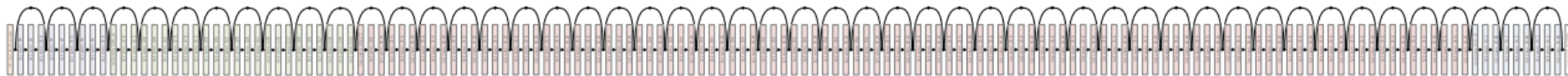
ResNet

- Overfitting?
 - No. Not only testing but also **training** errors increase!
- This is a general phenomenon, observed in many datasets
- It is caused by **gradient exploding/vanishing**



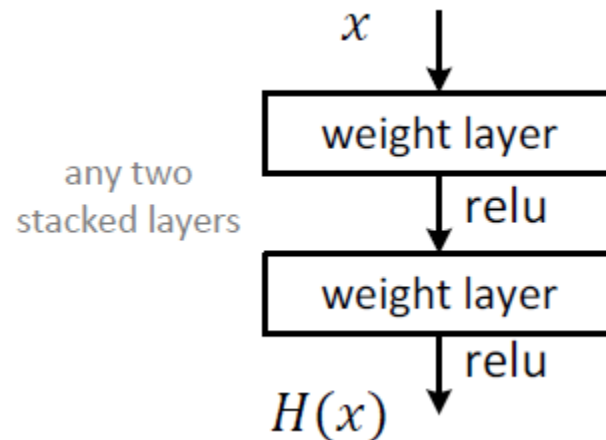
ResNet

- Architecture overview
 - 50, 101, 152 layers
 - Very deep, even more than 1,000 layers
 - It is constructed by stacking **residual** modules
 - Superior performances



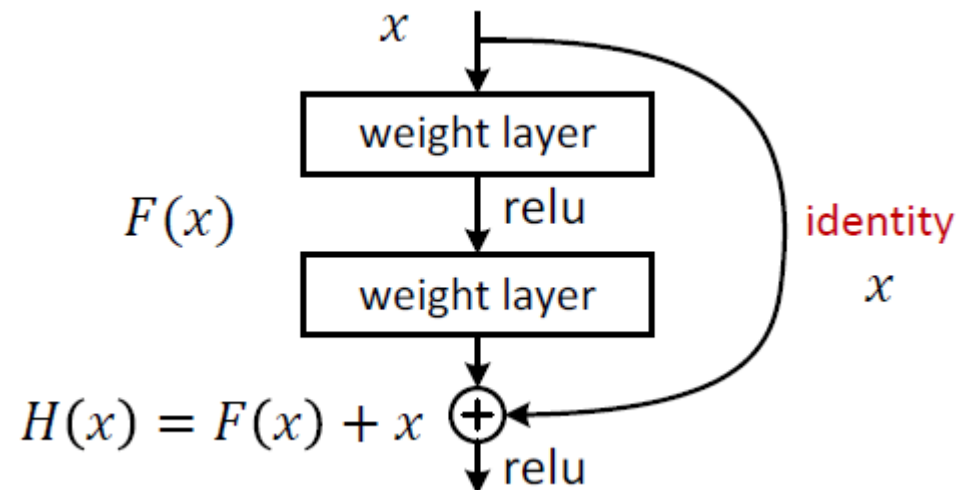
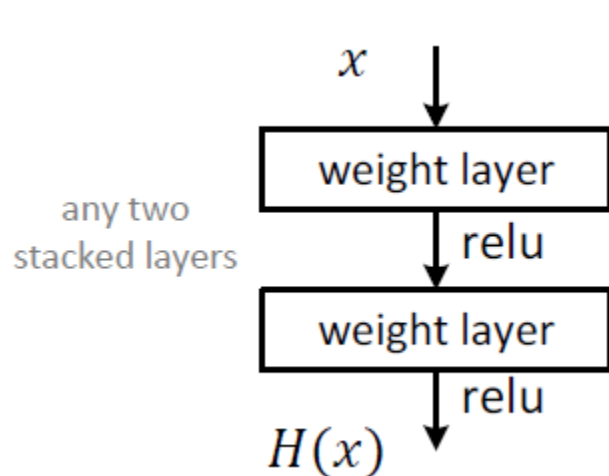
ResNet: Residual module

- The amount of changes (output - input) is fixed
- Adding more layers makes changes between two adjacent layers smaller
- Optimal mappings are **close** to **identity**
- Difficult to approximate an identity mapping due to ReLU



ResNet: Residual module

- Learn the **residual** $F(x)$, instead of the desired output $H(x)$
- Residual: Difference between the input and the desired output
- **Shortcuts connections**



ResNet: Residual module

- Learn the **residual** $F(x)$, instead of the desired output $H(x)$
- Residual: Difference between the input and the desired output
- **Shortcuts connections**

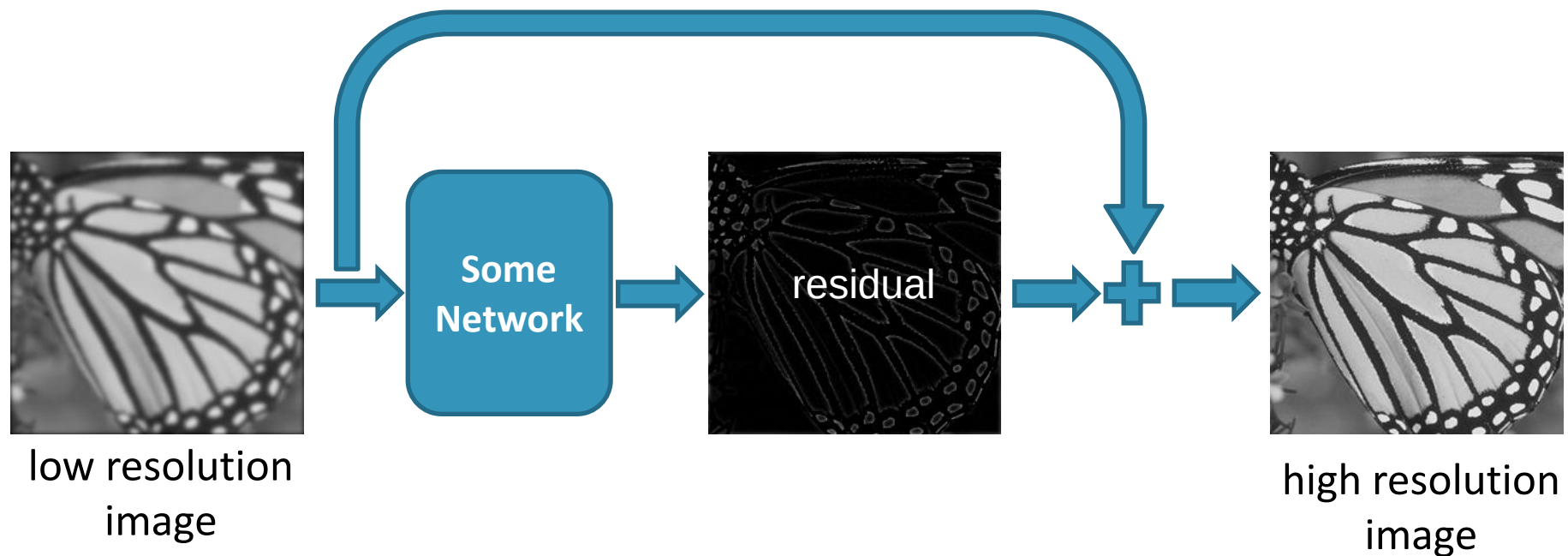
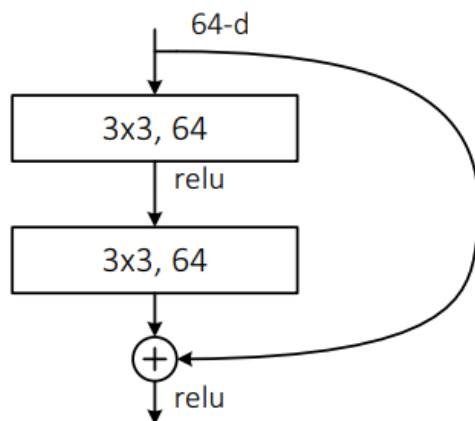


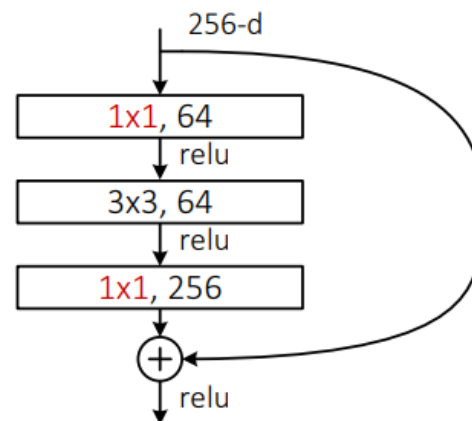
photo: H. Son

ResNet: Residual module

- A practical way to go deeper
- Inexpensive 1x1 convolutional layers for channel reduction



all-3x3



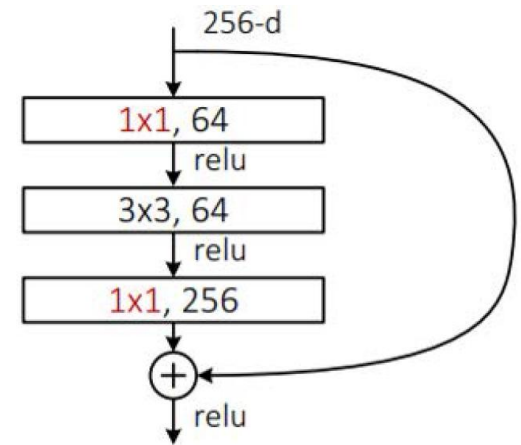
bottleneck

(for ResNet-50/101/152)

photo: H. Son

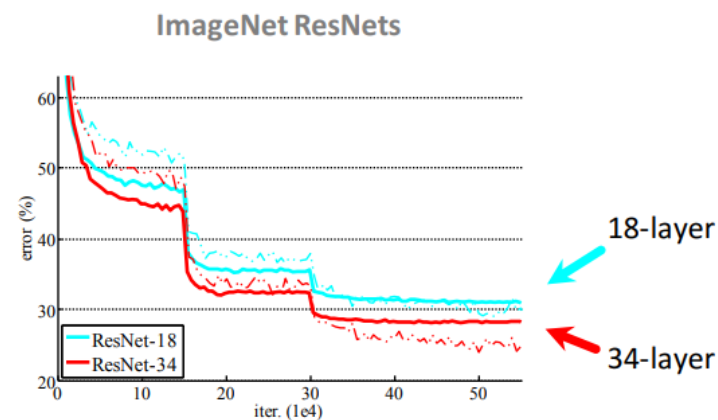
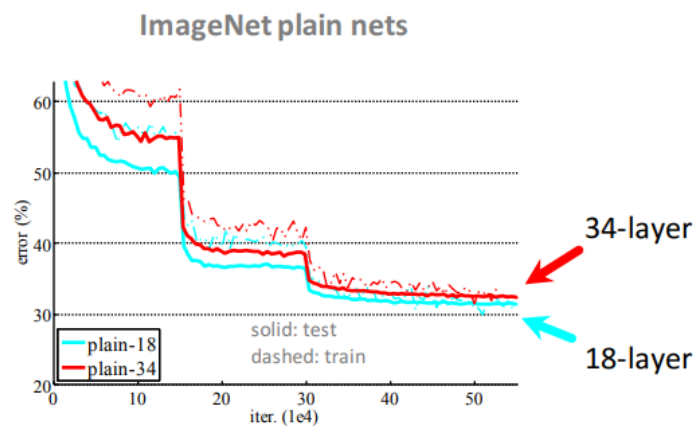
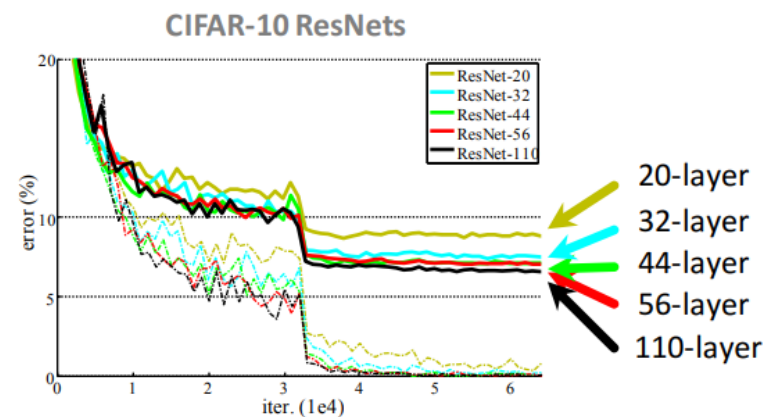
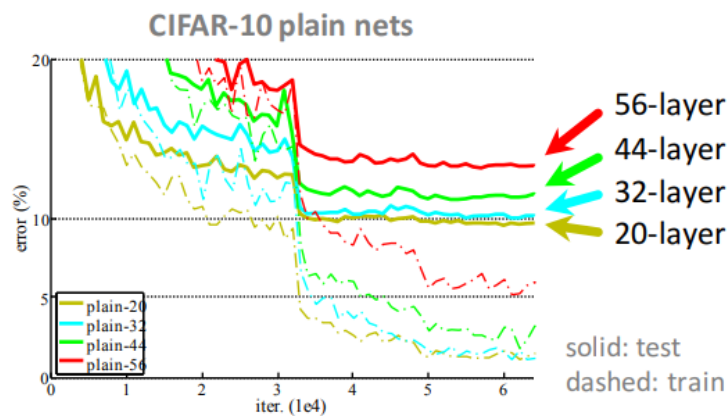
ResNet: Residual module

- Less parameters in a single module
- Accelerate the training of deep networks
- Reduce the risk of vanishing gradient
- Increase depth of the network
- Achieve higher accuracy in many vision applications



Experimental results: Convergence

- Convergence

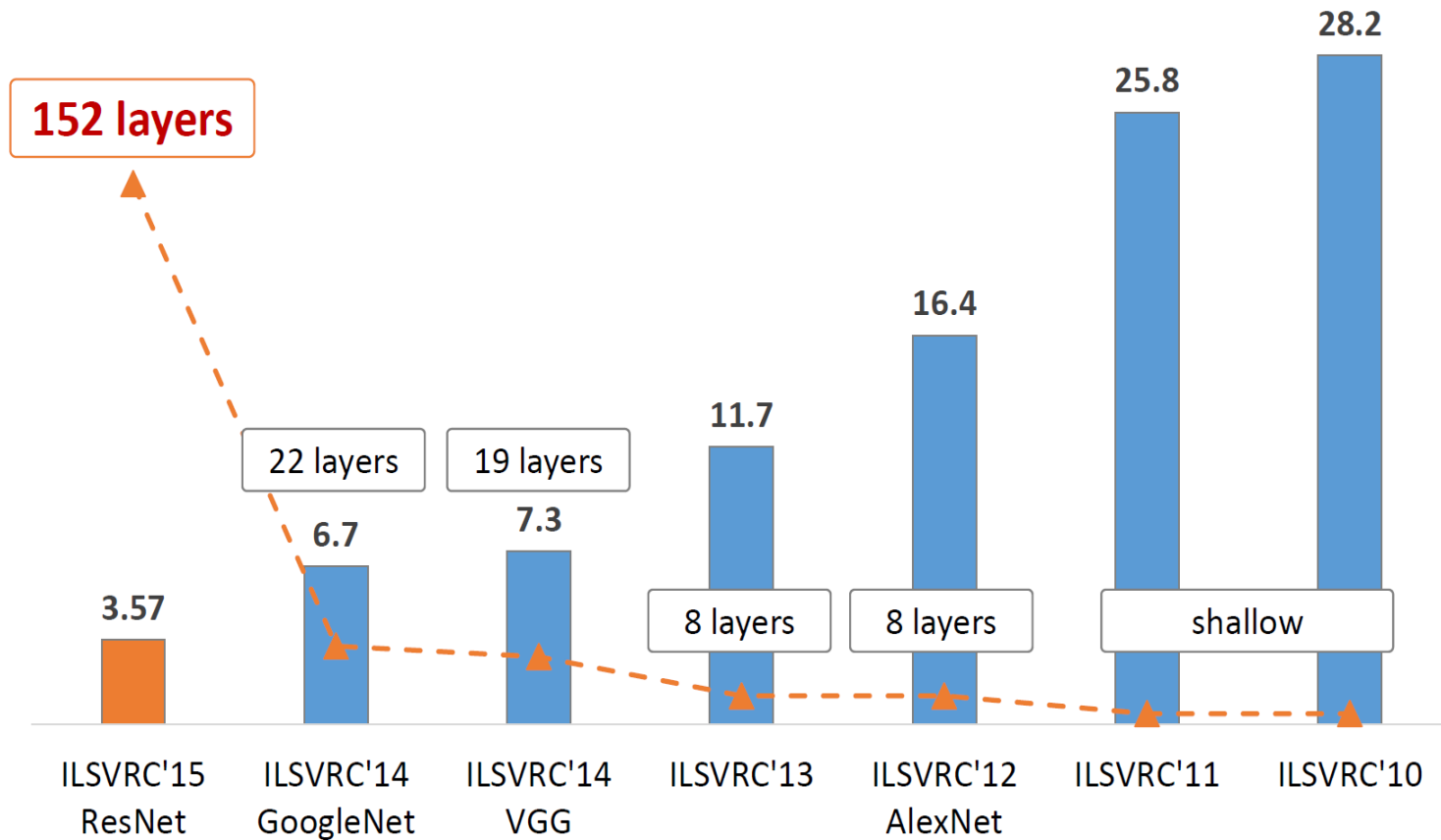


Experimental results: Comparison with SOTA

method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57



Experimental results: Depth vs. Error



slide: He et al.



Revolution of depth

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, 152 layers
(ILSVRC 2015)



slide: He et al.



Experimental results on more datasets and applications

- Applications: localization, detection, segmentation, ...
- Datasets: ImageNet, MS COCO, ...

task	2nd-place winner	MSRA	margin (relative)
ImageNet Localization (top-5 error)	12.0	9.0	27%
ImageNet Detection (mAP@.5)	53.6	62.1	16%
COCO Detection (mAP@.5:.95)	33.5	37.3	11%
COCO Segmentation (mAP@.5:.95)	25.1	28.2	12%

**absolute
8.5% better!**

photo: He et al.



Experimental results: Object detection

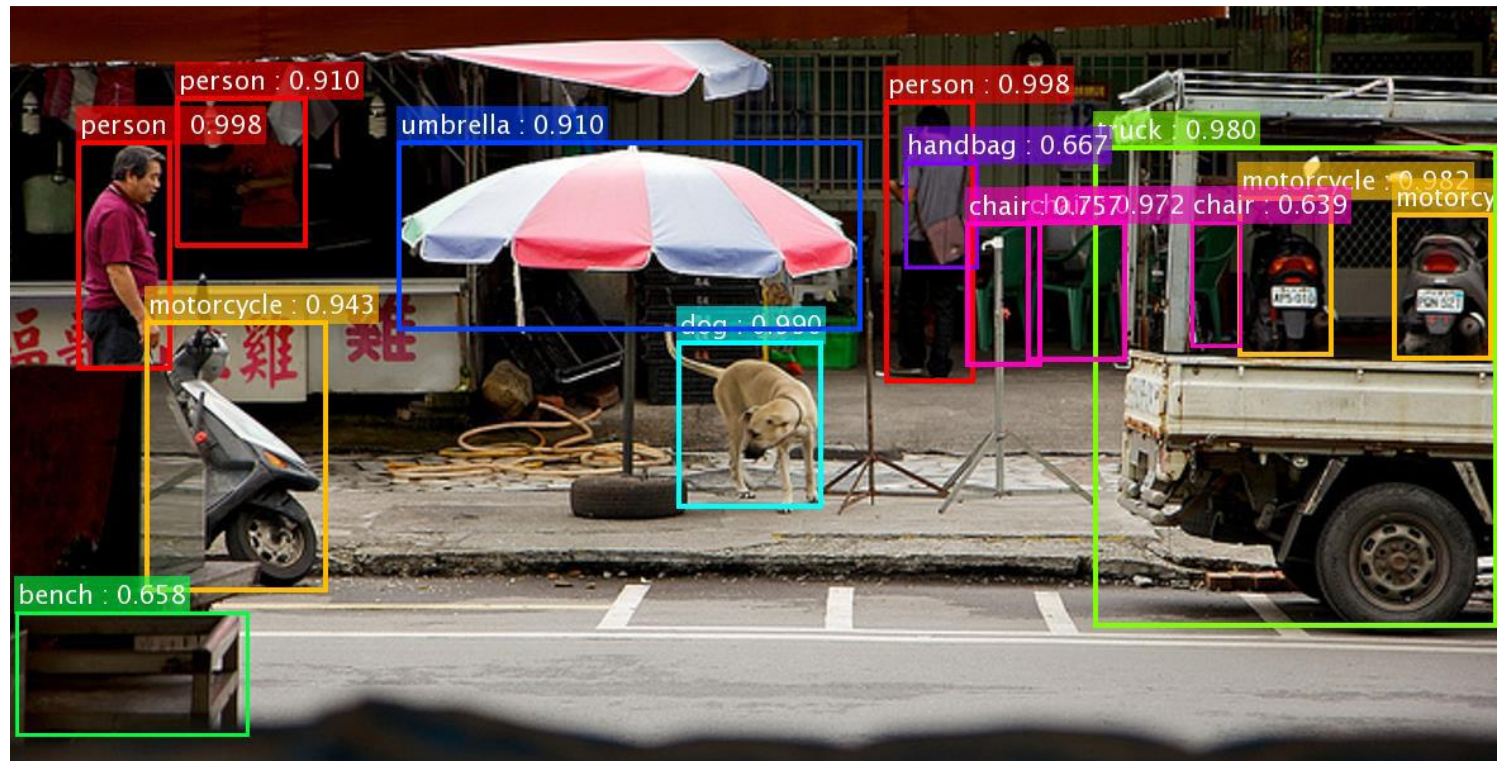


photo: He et al.



Experimental results: Instance segmentation

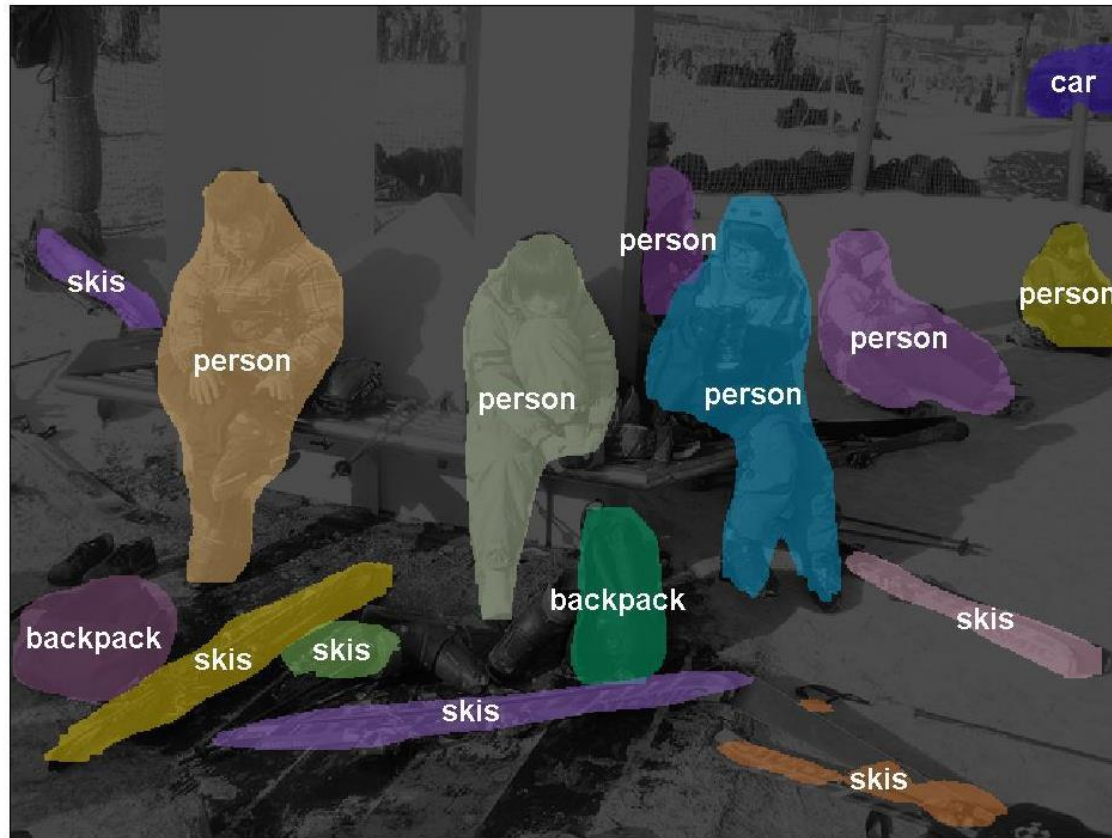


photo: He et al.



Outline

- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs

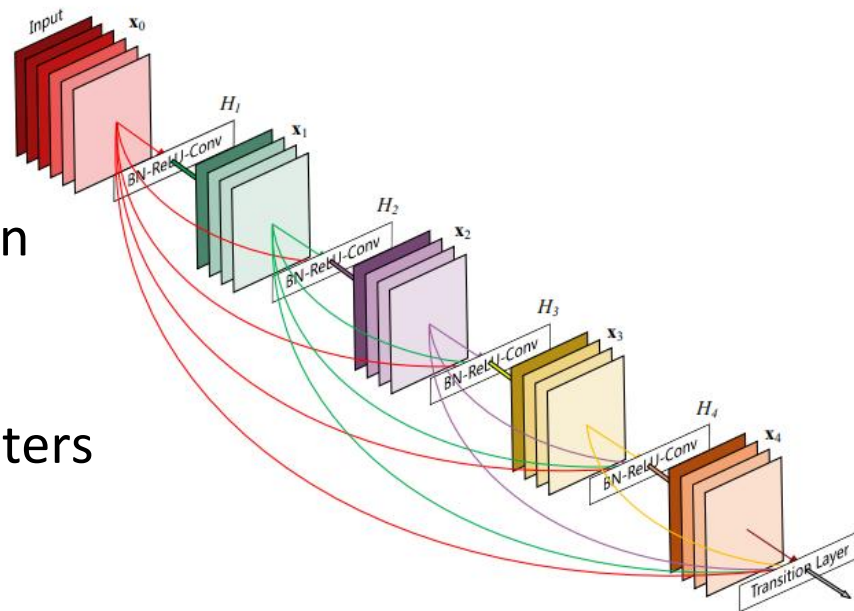


DenseNet

[Huang et al., CVPR'17]

Best Paper Award!

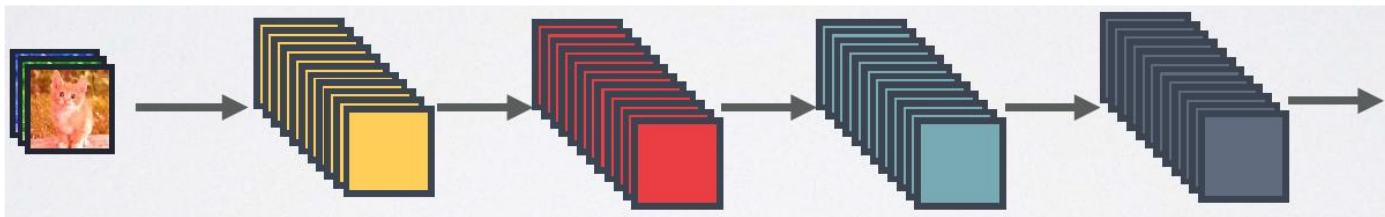
- Network architecture
- Densely connect each layer to every other layer
- For a layer, feature maps of all preceding layers are its input
- Advantages:
 - Alleviate vanishing gradient
 - Strengthen feature propagation
 - Encourage feature reuse
 - Reduce the number of parameters



CNNs vs. ResNet

- Conventional CNNs

- A convolutional layer takes feature maps from its preceding layer



- ResNet

- One additional path (identity mapping) with element-wise addition

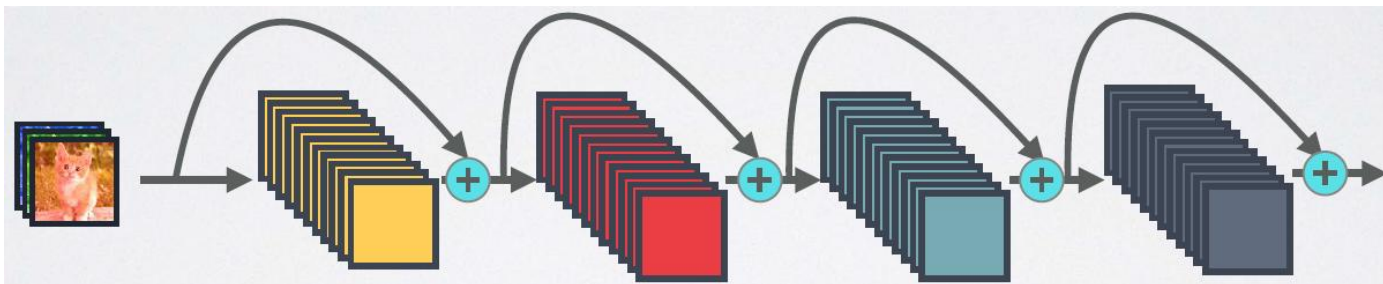


photo: Huang et al.

DenseNet

- Dense connectivity: a link between every layer pair
 - Each layer takes all preceding feature maps as input by **channel-wise concatenation**
 - Reduce the risk of gradient vanishing
 - Feature reuse
 - Fewer filters per layer
 - **High computation cost** and **more parameters per filter**?

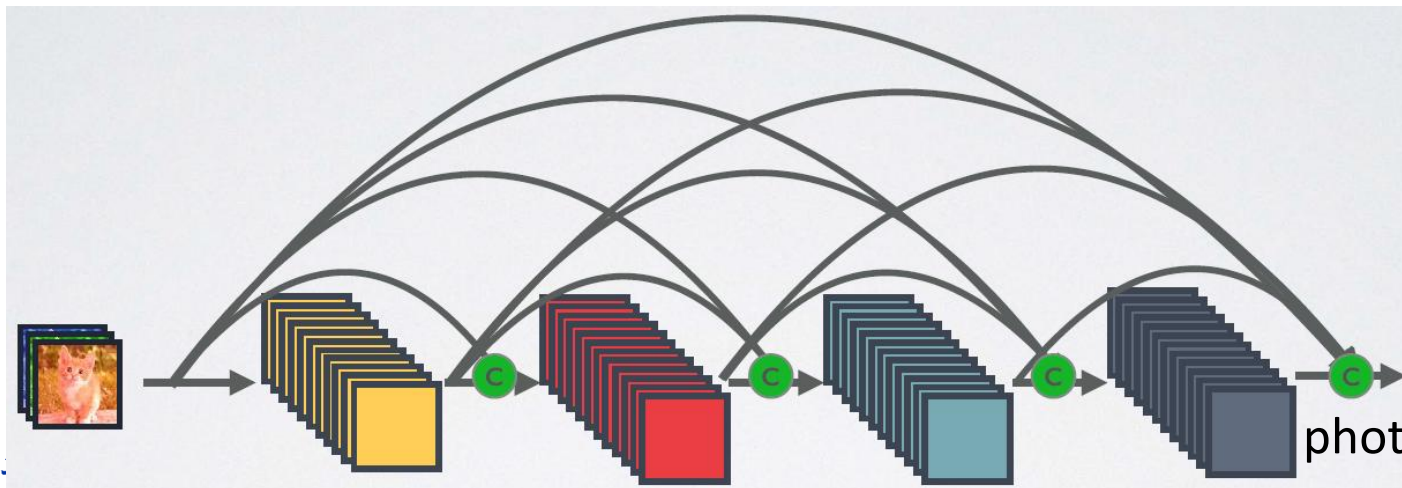


photo: Huang et al.

DenseNet

- 1x1 convolutional layer for channel (feature map) reduction
- Suppose each layer produces k feature maps
- There will be $l \times k$ channels at the l th convolutional layer
- Reduce to $4 \times k$ channels in advance
- No pooling for map size reduction?

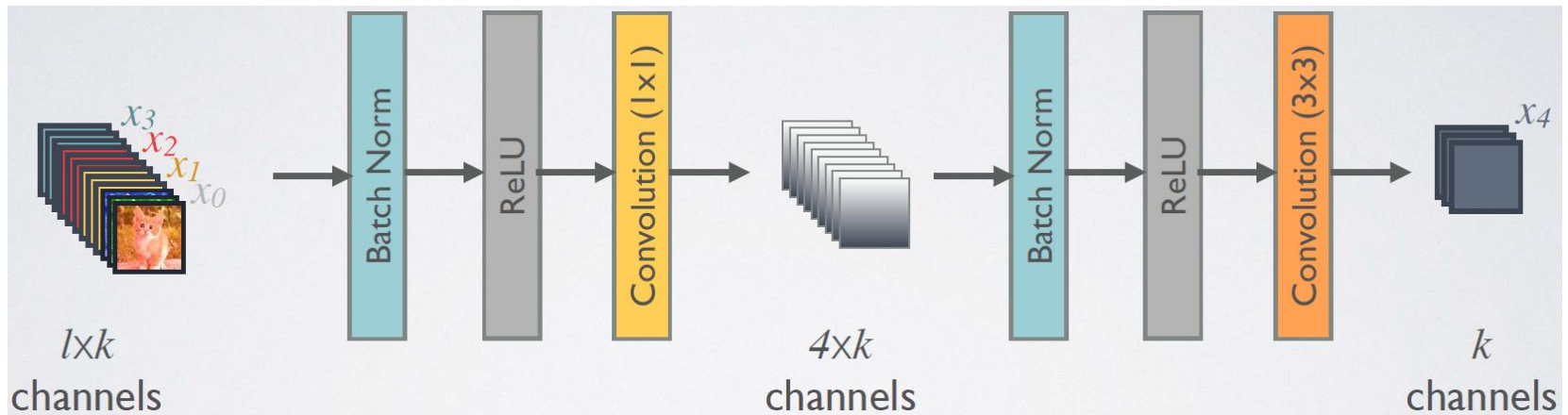


photo: Huang et al.

DenseNet

- Dense block: a few densely connected convolutional layers
- Stack dense blocks
- Insert a convolutional layer and a pooling layer into two connected blocks

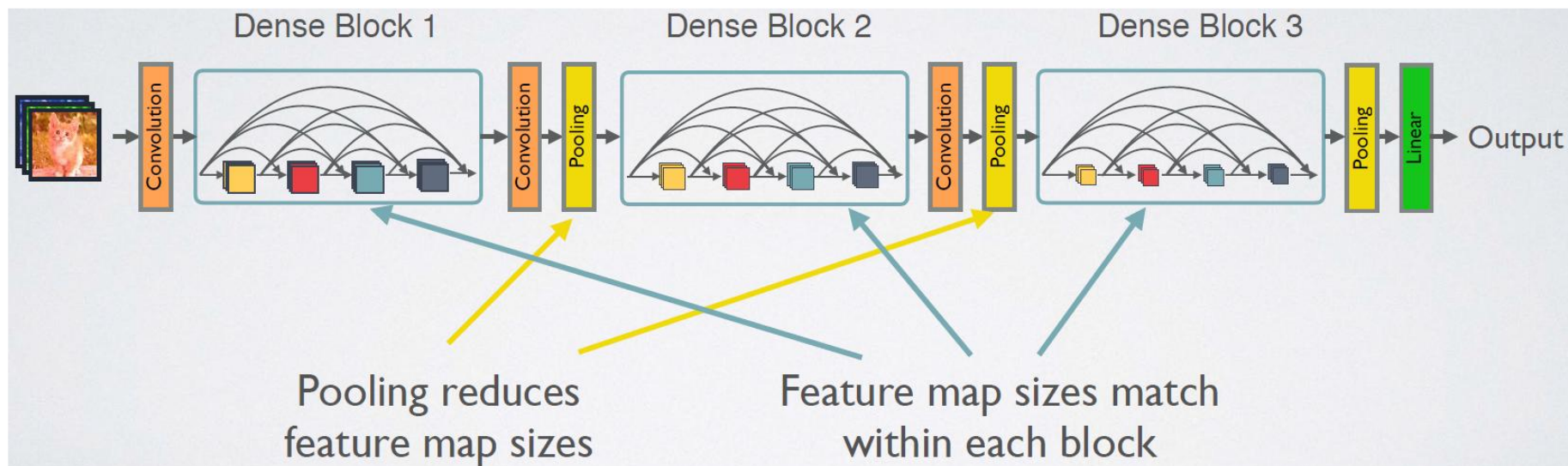
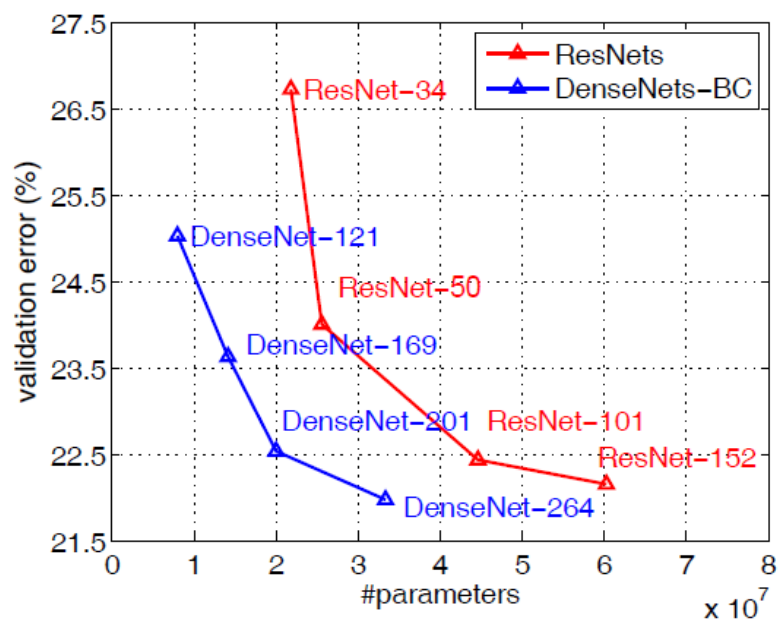
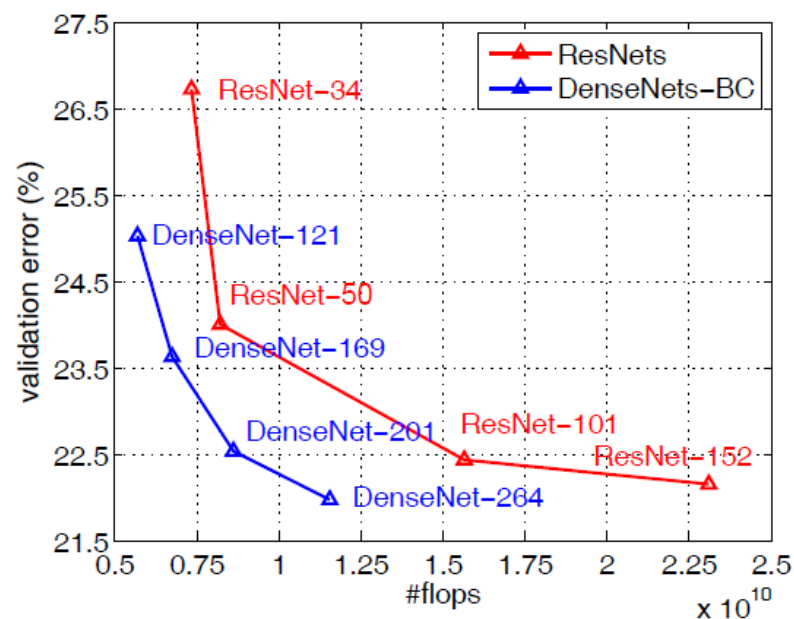


photo: Huang et al.

Experimental results: Comparison with ResNet



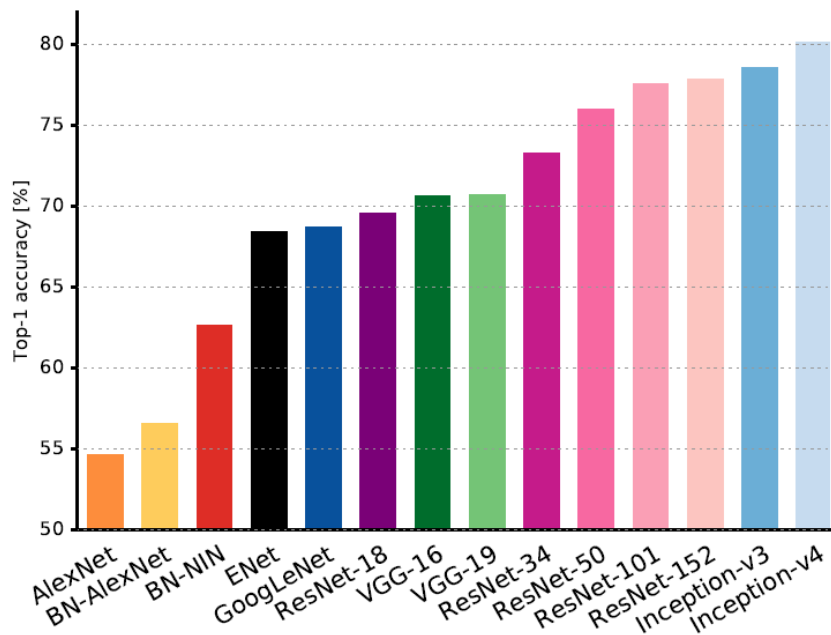
error vs. #parameters



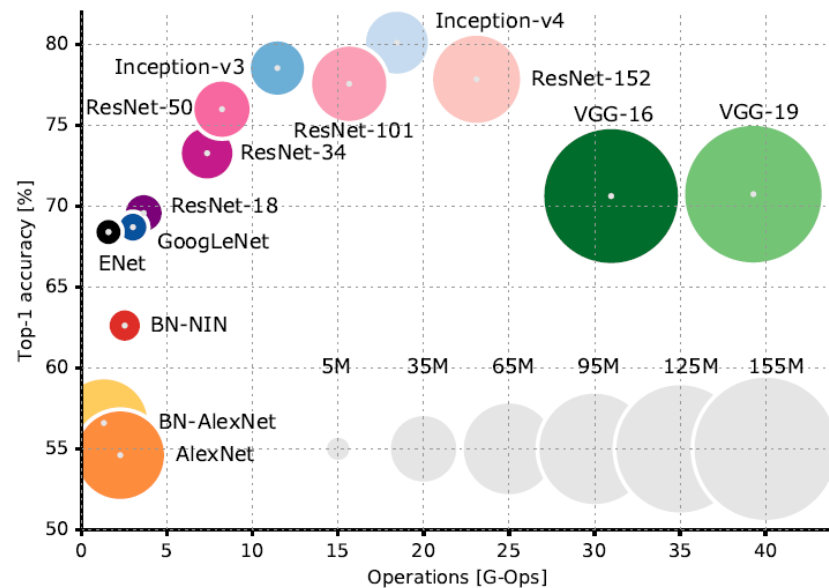
error vs. #flops

Performance comparison

[Canziani et al., arXiv'17]



top-1 acc. vs. network



top-1 acc. vs. #operations
with #parameters



Outline

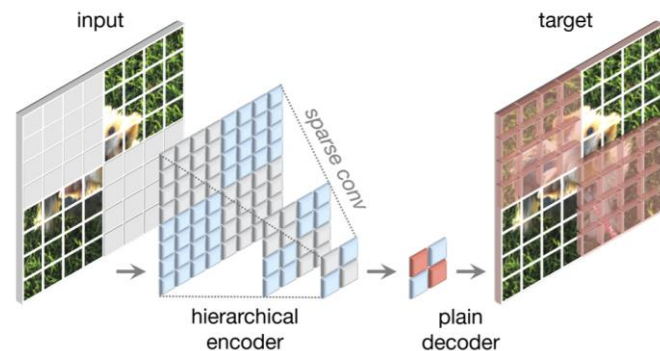
- LeNet-5
- AlexNet
- VGGNet
- GoogleNet
- ResNet
- DenseNet
- Advanced CNNs



ConvNeXt V2

[Woo et al., CVPR'23]

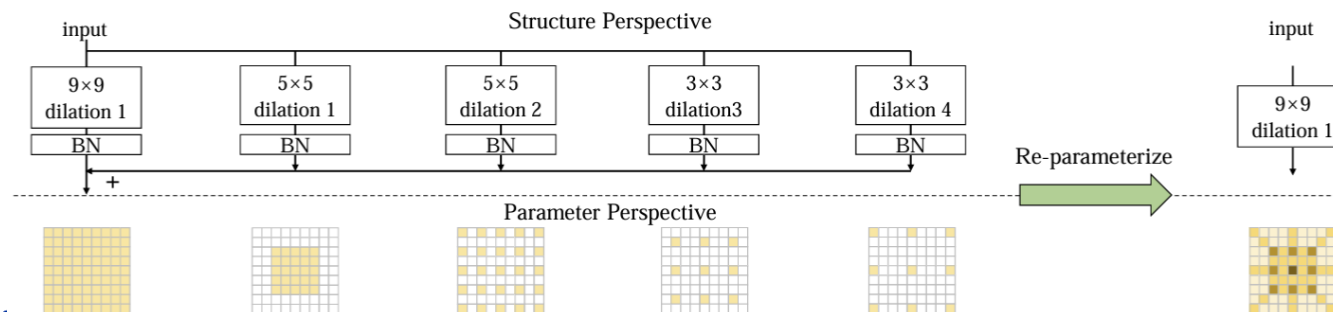
- A pure convolutional neural network, co-designed to excel in **self-supervised learning (SSL)** to achieve SOTA performance
- Key architectural improvements from V1 to V2
 - **Global response normalization (GRN)**: Address the issue of **feature collapse** (i.e., dead or redundant channels)
 - **Removal of LayerScale**
 - **Fully convolutional masked autoencoder (FCMAE)**: Employ **sparse convolutions** for visible image patches and prevent **information leakage** from masked areas



UniRepLKNet (Universal RepVGG-style Large Kernel Network)

[Ding et al., NeurIPS'23]

- UniRepLKNet leverages **large kernels** to create a universal backbone that competes with Vision Transformers (ViTs)
- Key features:
 - **Large kernels**: It uses larger convolutional filters to achieve a global receptive field
 - **Structural re-parameterization**: It uses multiple branches (including small kernels) to capture fine details
 - **Dilated Reparam Block**: A specialized structure that uses dilated convolutions to expand the receptive field further



Other advanced CNNs

- **MobileNetV4** (arXiv 2024): An architecture specifically optimized for mobile NPUs. It introduces the Universal Inverted Bottleneck (UIB), which allows the model to be adapted to various hardware accelerators.
- **ShiftwiseConv** (CVPR 2025): It emulates the expansive receptive fields of large kernels using significantly smaller kernels, thereby drastically reducing the parameter footprint.



References

- **LeNet-5**
 - Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner: Gradient-Based Learning Applied to Document Recognition, Proceeding of IEEE, 1998.
- **AlexNet**
 - A. Krizhevsky, I. Sutskever, and G. Hinton: ImageNet Classification with Deep Convolutional Neural Networks, in NIPS, 2012.
- **VGGNet**
 - K. Simonyan & A. Zisserman: Very Deep Convolutional Networks for Large-Scale Image Recognition, in ICLR, 2015.
- **GoogleNet**
 - C. Szegedy et al.: Going Deeper With Convolutions, in CVPR, 2015.
- **ResNet**
 - K. He et al.: Deep Residual Learning for Image Recognition, in CVPR, 2016.
- **DenseNet**
 - G. Huang et al. Densely Connected Convolutional Networks, in CVPR 2017.



References

- **ConvNeXt V2**

- S. Woo, S. Debnath, R. Hu, X. Chen, Y. Liu, K. He, and S. Xie: ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders, in CVPR, 2023.

- **UniRepLKNet**

- J. Ding, C. Han, G. Zhang, X. Luo, G. Cheng, and J. Wang: UniRepLKNet: A Universal Perception Control Network for Large-Scale Representation Learning, in NeurIPS, 2023.

- **MobileNetV4**

- D. Qin, C. Leichner, M. Delakis, M. Fornoni, S. Luo, F. Yang, W. Wang, C. R. Banbury, C. Ye, B. Akin, V. Aggarwal, T. Zhu, D. Moro, and A. Howard: MobileNetV4 -- Universal Models for the Mobile Ecosystem, in ECCV, 2024.

- **ShiftwiseConv**

- D. Li, L. Li, Z. Chen, and J. Li: ShiftwiseConv: Small Convolutional Kernel with Large Kernel Effect, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, in CVPR, 2025.



Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

Yen-Yu Lin (林彥宇)

Email: lin@cs.nycu.edu.tw

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

